PES University · Capstone Project (UE23CS441A)

CETP

Cross-Environment Runtime Prediction for PostgreSQL Queries

Master Technical Reference

Architecture, implementation, evidence, limitations and defense preparation

Document version	1.0 (first complete edition)
Prepared	27 October 2026
Repository state	branch `x86-probe`, HEAD `38c4bf2`, working tree modified
Software version	`cetp` 0.4.0 (classifier "Alpha")
Project team	Harshith J, Ishanvee Amit Sinha, Sarthak Wadhwa, Kartik Bhat Sumbly
Guide	Prof. Nivedita Kasturi
Status	Research prototype; evidence-based, not a marketing document

Every quantitative claim in this document carries a source path and an evidence tag (see the Evidence methodology section above). Where the repository contradicts itself, the contradiction is reported rather than resolved silently.

Project status and document scope

Project status. CETP is an academic capstone prototype. It predicts how long a PostgreSQL query will take on a *production* machine from an `EXPLAIN (ANALYZE)` captured on a *development* machine plus a description of each machine's capability. The repository contains two generations of the method (Section 1.6); the installable package is version 0.4.0 and is classified "Alpha" in `pyproject.toml`. The repository's own README states: "The research is sound; treat the product as alpha." This document agrees with the first half of that sentence only in a restricted sense, spelled out in Chapters 1 and 17.

What this document covers. The state of the repository on 27 October 2026 (HEAD `38c4bf2`, branch `x86-probe`, 17 commits, working tree modified), including the uncommitted fix of evidence-integrity finding C10 described in Section 17.3.

What it does not cover. (i) Any measurement taken after that date. (ii) Content of the ca. 50 PDF reports under `docs/` (reorganised into `docs/{design,guides,reference,results}` in the uncommitted working tree). They were not read; in the audit pass I extracted text from seven of them (CANONICAL, operator_model, laptop_to_prod, cross_isa_prediction, HANDBOOK, LLD, DATASHEET) only to look up specific numbers, and equivalence with their deleted Markdown sources is unverified. (iii) The C sources of the Generation-1 hardware probe (only the header comment of `hw_probe_suite.c` was read) and the bodies of about 25 analysis scripts, which were not read in full. (iv) The original EC2 raw logs, which no longer exist in the repository. Each gap is tracked in Appendix I.

Contents

Project status and document scope i

List of abbreviations and terminology 1

Document purpose and intended audience 2

Evidence and verification methodology 3

1 Executive overview 4

1.1 What CETP is . . . 4
1.2 The honest headline . . . 4
1.3 What is genuinely contributed . . . 4
1.4 What it is not . . . 5
1.5 System at a glance . . . 5
1.6 Two generations, one repository . . . 5
1.7 Reading guide to the numbers . . . 6

2 Problem statement and requirements 7

2.1 The problem . . . 7
2.2 Why it is hard . . . 7
2.3 Scope: what is in and out . . . 7
2.4 Requirements . . . 7
2.4.1 Functional requirements . . . 7
2.4.2 Non-functional requirements . . . 8
2.4.3 Success criteria . . . 8
2.5 The central trade-off . . . 9

3 Background and related work 10

3.1 Concepts a reader needs . . . 10
3.1.1 How PostgreSQL reports time . . . 10
3.1.2 Why parallelism dominates cross-machine differences . . . 10
3.1.3 Metrics . . . 10
3.1.4 Cross-validation protocols . . . 10
3.2 Related work . . . 11

4 System architecture 12

4.1 Architectural style . . . 12
4.2 Component view . . . 12
4.3 Runtime data flow for one prediction . . . 12
4.4 Deployment topology . . . 13
4.5 Design principles visible in the code . . . 13

5 Technology stack **14**
5.1 Languages and runtimes . . . 14
5.2 Libraries and pinned versions . . . 14
5.3 Choices a reviewer will ask about . . . 15
5.4 Dependency risks . . . 15

6 Repository walkthrough **16**
6.1 Top-level layout . . . 16
6.2 The package: module by module . . . 17
6.3 Where the evidence lives . . . 17
6.4 Reading order for a new contributor . . . 17
6.5 Hazards in the tree . . . 18

7 Detailed implementation **19**
7.1 Plan decomposition . . . 19
7.2 The per-operator model . . . 20
7.3 Product flow and guards . . . 21
7.4 Privacy, ingestion and telemetry . . . 22
7.5 Regression detection . . . 23
7.6 Generation-1 predictor and gate . . . 24
7.7 Acquisition: the operator probe . . . 24

8 Algorithms and mathematical models **26**
8.1 Generation 1: two-scalar analytical model . . . 26
8.1.1 Definition . . . 26
8.1.2 Why it is called “roofline” and why that is a stretch . . . 26
8.1.3 Hand-checked example . . . 26
8.1.4 Confidence . . . 26
8.2 Generation 2: per-operator model . . . 27
8.2.1 Derivation . . . 27
8.2.2 Cardinality factor . . . 27
8.2.3 Concurrency . . . 27
8.2.4 Few-shot calibration . . . 27
8.3 Uncertainty: conformal multipliers and quantile regression . . . 28
8.3.1 Split-conformal on log residuals . . . 28
8.3.2 Gradient-boosted quantile regression . . . 28
8.4 Evaluation algorithms . . . 28
8.5 Complexity . . . 29

9 End-to-end workflows **30**
9.1 W1. Reproduce the published numbers . . . 30
9.2 W2. Capture a machine signature . . . 30

9.3 W3. Offline prediction from two plan files (`cetp predict-prod`) . . . 30
9.4 W4. Live prediction with plan forcing (`predict_prod.py`) . . . 30
9.5 W5. CI gate (Generation 1) . . . 31
9.6 W6. Ingestion and calibration loop . . . 31
9.7 W7. Container grid collection . . . 31
9.8 A worked example: one real execution, step by step . . . 31
9.9 The CLI bypasses the cross-architecture refusal . . . 32

10 Data, schemas and storage **33**
10.1 Datasets . . . 33
10.1.1 Machine inventory . . . 33
10.2 Schemas . . . 33
10.2.1 Operator-signature registry record . . . 34
10.2.2 Calibration store . . . 34
10.2.3 Telemetry (JSON Lines) . . . 34
10.2.4 Gen-1 conformal table . . . 34
10.2.5 Evaluation outputs . . . 34
10.3 Data flow and lineage . . . 34
10.4 Integrity controls . . . 34

11 Interfaces and APIs **36**
11.1 Command-line interface . . . 36
11.2 Python API (public surface) . . . 37
11.3 Result dictionary (Gen 2) . . . 37
11.4 Gate interface . . . 37
11.5 CI and editor integrations . . . 37
11.6 Error handling summary . . . 38

12 Testing and verification **39**
12.1 Test inventory . . . 39
12.2 Test strategy . . . 39
12.3 Coverage versus requirements . . . 39
12.4 Verification beyond unit tests . . . 40
12.5 Sanity checks on results (mine) . . . 40

13 Results and evaluation **41**
13.1 Study A: five EC2 machines, TPC-H SF1 (Gen 1) . . . 41
13.1.1 Per machine (LOMO) . . . 41
13.1.2 Classifier and noise . . . 41
13.2 Study B: Docker capability grid (Gen 2) . . . 42
13.3 Study C: laptop to “production” (242 pairs, 11 targets) . . . 42
13.4 Study D: laptop to EC2 (95 pairs) . . . 43

13.5 Study E: uncertainty and gating . . . 43
13.6 Study F: concurrency, calibration loop, drift . . . 43
13.7 Summary of what the evidence supports . . . 44

14 Setup, deployment and operation **45**
14.1 Prerequisites . . . 45
14.2 Install and smoke test . . . 45
14.3 Typical tasks . . . 45
14.4 Configuration . . . 46
14.5 Operations . . . 46
14.6 Troubleshooting . . . 46

15 Security, reliability and performance **47**
15.1 Security and privacy . . . 47
15.1.1 Threat model (as implemented) . . . 47
15.2 Reliability . . . 47
15.3 Performance . . . 48
15.4 Scalability . . . 48
15.5 Compliance and licensing . . . 48

16 Design decisions, trade-offs and alternatives **49**
16.1 Trade-offs that shaped the project . . . 50
16.2 Alternatives considered and not taken . . . 50
16.3 What I would change first . . . 50

17 Limitations, risks and future work **51**
17.1 Limitations that bound the claims . . . 51
17.2 Known defects and inconsistencies . . . 51
17.3 Evidence-integrity finding C10 and its fix . . . 52
17.4 Risks . . . 52
17.5 Future work . . . 52
17.5.1 Immediate (hours to days) . . . 52
17.5.2 Medium term (weeks) . . . 52
17.5.3 Long term (research) . . . 53

18 Conclusion **54**
18.1 What was built . . . 54
18.2 What the evidence shows . . . 54
18.3 Answer in 30 seconds . . . 54
18.4 Answer in two minutes (technical) . . . 54
18.5 Honest overall assessment . . . 54

A Glossary **56**

B Acronyms **57**

C Function and module reference **58**

D Configuration reference **59**

E Data schemas **60**

F Reproducibility commands **61**

G Requirement-to-implementation traceability **62**

H Results and evidence table **63**

I Known issues and contradictions register **64**

I.1 Defects found or confirmed in this audit 64

I.2 Evidence-pack register 65

I.3 Not done 65

J Viva question bank **66**

Appendix K References and bibliography **68**

L Independent audit report **69**

L.1 Audit table 70

L.2 Missing topics 72

L.3 Contradictions between chapters (found and resolved) 72

L.4 Unsupported or only partly supported claims that remain 72

L.5 Important implementation details previously undocumented 72

L.6 Questions the project owner must answer 72

L.7 Compilation and reference check 72

List of Figures

1.1 Gen-2 (per-operator) prediction path. Derived from cetp/plan\unhbox\voidb@x\kern.04em\vbox{\hrulewidth.5emheight.4pt}\penalty0\hskip0pt\relaxnodes.py, cetp/operator\unhbox\voidb@x\kern.04em\vbox{\hrulewidth.5emheight.4pt}\penalty0\hskip0pt\relaxmodel.py and cetp/cross\unhbox\voidb@x\kern.04em\vbox{\hrulewidth.5emheight.4pt}\penalty0\hskip0pt\relaxenv.py. 5

4.1 Component view. Solid teal = Gen 2 (per-operator); dashed = Gen 1 (two-scalar). Arrows show the main data flow; they are *not* Python imports. Checked against the import statements (audit pass): cross_env imports operator_model and plan_nodes; operator_model imports plan_nodes; calibration_store imports operator_model; predict and regress import signature; ingest imports redact; telemetry, redact, plan_nodes and signature import nothing from the package; the gate imports none of them. 12

4.2 Sequence for one Gen-2 prediction. The cross-architecture refusal fires inside cross_env.predict only if the caller passed the machine records (see Section 9.9). 13

9.1 W4, from scripts/collection/predict_prod.py. Red boxes touch production or the measured truth. Only step 1 contacts production unless -actual-ms is a real measurement. 30

10.1 Lineage of data to published numbers. Only the right half is re-runnable from a fresh clone. 34

13.1 Leave-one-machine-out MAPE, 840 held-out evaluations (results/canonical\unhbox\voidb@x\kern.04em\vbox{\hrulewidth.5emheight.4pt}\penalty0\hskip0pt\relaxmetrics.json, [FILE]). 41

13.2 Eight arm64 container cells, 22 queries, 1,232 pairs. "par" = parallel-speed-up term enabled. (results/operator\unhbox\voidb@x\kern.04em\vbox{\hrulewidth.5emheight.4pt}\penalty0\hskip0pt\relaxmodel\unhbox\voidb@x\kern.04em\vbox{\hrulewidth.5emheight.4pt}\penalty0\hskip0pt\relaxmetrics.json, [FILE].) 42

13.3 Dev machine: Apple-M1 laptop. Containers run on the same host; "gha-*" are x86_64 runners (results/laptop\unhbox\voidb@x\kern.04em\vbox{\hrulewidth.5emheight.4pt}\penalty0\hskip0pt\relaxto\unhbox\voidb@x\kern.04em\vbox{\hrulewidth.5emheight.4pt}\penalty0\hskip0pt\relaxprod\unhbox\voidb@x\kern.04em\vbox{\hrulewidth.5emheight.4pt}\penalty0\hskip0pt\relaxmetrics.json, [FILE]). 42

List of Tables

13.1 Parallel speed-up π per container cell (probe; `models/operator\unhbox\voidb@x\kern.04em \vbox{\hrulewidth.5emheight.4pt}\penalty0\hskip0pt\relaxsignature.json`). . . 42

List of abbreviations and terminology

Term	Meaning in this document
AURC	Area under the risk–coverage curve; average error when the most confident predictions are kept first.
CI	Confidence interval; here always a 95% percentile cluster-bootstrap interval unless stated.
Cluster bootstrap	Resampling whole cross-validation folds (machines) with replacement, so the 420 pairs are not treated as independent.
CPU quota (CFS)	Linux Completely Fair Scheduler limit on CPU time (`docker --cpus`); slows parallel work but not a single thread.
CV (timing)	Coefficient of variation of repeated timings, standard deviation divided by mean. Not cross-validation; that is always spelled out.
DOP	Degree of parallelism available to a query: $1 +$ `max_parallel_workers_per_gather` (3 at PostgreSQL defaults).
EBS	Amazon Elastic Block Store; network-attached storage used by all five EC2 machines.
EXPLAIN ANALYZE	PostgreSQL command that executes a query and returns its plan with actual per-node timings, row counts and buffer counts.
Gather	PostgreSQL plan node that launches parallel workers; every node below it runs once per participant.
GBR	Gradient-boosted regression trees (scikit-learn `GradientBoostingRegressor`).
GHA	GitHub Actions; its x86_64 runners supplied the cross-ISA target machines.
Gen 1 / Gen 2	The roofline-style two-scalar method and the per-operator method, respectively (Section 1.6).
ISA	Instruction-set architecture; here arm64 (Apple M1) versus x86_64.
LOMO, LOQO, LOMQO	Leave-one-machine-out, leave-one-query-out and leave-one-machine-and-query-out cross-validation.
MAPE, MdAPE	Mean and median absolute percentage error (Section 3.1.3).
Naive	The do-nothing baseline: predict that production runtime equals development runtime (scaling factor $= 1$).
Naive-linear	Baseline scaling by $1/\text{bandwidth}$ ratio.
PMU	Hardware performance-monitoring unit; blocked on AWS Nitro instances, which motivated the DB-only scope.
pp	Percentage points.
q-error	$\max(\hat{y}/y, y/\hat{y})$; symmetric multiplicative error used in the database literature.
Roofline	Performance model bounding throughput by compute or memory bandwidth; the project's "roofline" is a weighted blend, not a true minimum of bounds.
Scaling factor	$s = T_{\text{prod}}/T_{\text{dev}}$; the quantity every predictor in this project estimates.
SF1	TPC-H scale factor 1 (about 1 GB of base data).
SLA	Service-level agreement; here a per-query latency budget in milliseconds.
STREAM	Memory-bandwidth microbenchmark (triad kernel) [14].
TPC-H	Decision-support benchmark of 22 analytical queries [20].
Warm-up repetition	The first of 20 repetitions of a measurement cell; discarded because it is a cold-cache run.

Document purpose and intended audience

This is a technical master reference, written so that one person can read it and then: explain the project to a beginner, a professor, an interviewer or a teammate; trace an input through the code to the printed prediction; defend each design decision with evidence; reproduce the headline numbers; and answer detailed follow-up questions, including the uncomfortable ones. It is deliberately not a brochure. The repository is unusually candid about its own negative results, and this document keeps that candour.

Reader	Start here
Beginner or examiner new to the topic	Chapters 1 to 3, then Chapter 18
Technical reviewer / professor	Chapters 4, 7, 8, 13 and 17
Teammate taking over the code	Chapters 6, 7, 12 and 14, Appendix C
Interviewer or viva panel	Chapters 1, 13 and 17, and the question bank in Appendix J
Anyone quoting a number	Appendix H first, then `results/CANONICAL.md`

Evidence and verification methodology

Authority order. Current code and verified execution output are authoritative for how the system behaves *today*. Result files (`results/*.json`, `results/*.csv`) are authoritative for what was measured. Prose in comments, README, PDFs or earlier chats is used only to explain intent and is never allowed to override the first two. Where prose and measurement disagree, the disagreement is reported (Appendix I).

Evidence tags. Each claim in a table or numbered list carries one of these tags.

Tag	Meaning
[RUN]	I executed it or recomputed it myself in this session (output seen).
[PACK]	Executed by the author of the audited evidence pack (`docs/research/`) and recorded there; not re-run by me.
[FILE]	Read from a committed result, data or configuration file; the computation behind it was not re-run.
[CODE]	Established by reading the source (path and line numbers given).
[CLAIM]	Stated in a comment, README or report but not traced to code or data.
[UNK]	Unknown or unavailable; the missing step is named.

What was actually done for this edition.

1. Read in full: all twelve modules of `cetp/` (about 2,100 lines), `gate/cetp_gate.py`, `scripts/collection/operator_probe.py`, the parts of `scripts/analysis/canonical_evaluation.py` and `operator_model_evaluation.py` that define data loading, predictors, metrics, cross-validation and bootstrap, `scripts/analysis/build_calibration.py`, the Makefile, `pyproject.toml`, `requirements.txt`, `Dockerfile`, the README and the three CI workflows (headers).
2. Read the ten evidence-pack files `docs/research/00--09*.md` and the committed result files cited in Appendix H.
3. **[RUN]** `python3 -m cetp.cli selftest` printed `selftest PASSED` (5 checks) in a Python 3.10.12 Linux environment with no third-party packages.
4. **[RUN]** `pytest tests/test_operator_model.py tests/test_product_loop.py`: **80 passed** in 0.10 s (same environment). The other two suites need scikit-learn, numpy and pandas and were not run by me.
5. **[RUN]** Decomposed and priced the three real EXPLAIN fixtures (`q1`, `q18`, `q19`) with the shipped code and the committed operator registry (Section 9.8).
6. **[RUN]** Recomputed pooled conformal coverage from `results/tier1_conformal.csv` (Section 13.5).
7. Searched the web to confirm that six of the cited database papers exist with the stated venues.

What was not done. No PostgreSQL server, Docker daemon or EC2 instance was used; no probe or collection script was run; `make results` and `make verify-determinism` were not run by me (the evidence-pack author ran `canonical_evaluation.py` and found its outputs byte-identical to the committed ones, **[PACK]**). The full 119-test suite result is **[PACK]**. The Generation-1 C probe source and the Chrome-based PDF build were not exercised. The bibliography was written from knowledge of the literature; only the six papers named above were looked up.

Honest reading

Every limitation above is an *unverified* item, not a failed one. Nothing in this document claims that an unrun command succeeds. Commands in Chapter 14 are labelled **[RUN]**, **[PACK]** or "inferred".

1 Executive overview

1.1 What CETP is

CETP (Cross-Environment Runtime Prediction) answers one question for a PostgreSQL user: *a query ran in X ms on my development machine; how long will it take on production?* The input is the `EXPLAIN (ANALYZE, FORMAT JSON)` of the query on the development machine, a numeric description of both machines, and (optionally) a handful of timed calibration queries on production. The output is a scaling factor $s = T_{\text{prod}}/T_{\text{dev}}$, a predicted runtime, and a trust level. Prediction itself needs no database access. The only step that writes to a database is the optional capability probe, which creates and then drops three `UNLOGGED` tables named `_cetp_probe_*` (Chapter 7). The package has no runtime dependencies (`pyproject.toml`, **[FILE]**).

1.2 The honest headline

The repository contains two generations of the method and one cannot summarise CETP without saying what each does and where each fails. The following statements are the ones I would defend to a panel.

Key finding

1. On the original five-machine EC2 study, a two-scalar analytical model beats the do-nothing baseline by a small, statistically supported margin. Leave-one-machine-out MAPE: naive 31.96%, naive-linear 23.05%, analytical 21.61%. The paired difference naive-linear minus analytical is +1.44 pp with 95% cluster-bootstrap CI [0.32, 2.56] **[FILE]** (`results/canonical_metrics.json`). The analytical model is *not* distinguishable from the even simpler compute-only rule: difference +1.55 pp, CI [−0.41, 3.51].

Key finding

2. A per-operator model that is aware of PostgreSQL parallelism wins decisively over scalar scaling where parallelism changes between machines. On the regenerated Docker grid (8 arm64 cells, 22 queries, 1,232 pairs) MAPE falls from 66.97% (scalar, geometric fallback) to 30.15% once the *parallel* scaling is added and to 22.41% for the full per-operator model; the naive baseline is 72.52% **[FILE]** (`results/operator_model_metrics.json`). The mechanism is that a parallel scan sees a speed-up of about 2.7× when the CPU quota allows three workers, and about 0.77× when it does not.

Caution

3. Zero-shot cross-ISA prediction currently fails. Predicting x86_64 GitHub-Actions runtimes from an arm64 laptop, the operator model has MAPE 66–73% against 21–23% for the naive baseline (mean 68.92% vs 21.74%) **[FILE]** (`results/laptop_to_prod_metrics.json`). Over all 11 targets the operator model is worse than doing nothing (43.72% vs 31.70%); only on the 7 same-ISA targets is it better (29.31% vs 37.39%). The library function therefore *refuses* to predict across architectures without calibration (`CrossArchitectureUncalibrated`, **[CODE]** `cetp/cross_env.py`); but the `cetp predict-prod` command does not pass the information the refusal needs and returns a prediction anyway (**[RUN]**, Section 9.9).

Caution

4. Laptop-to-EC2 transfer does not work for either method. On 95 pairs, naive error is 129.39% and the cheaper laptop-derived signatures are far worse (160.68% and 337.75%) **[FILE]** (`results/laptop_to_ec2_metrics.json`). The operator model cannot be evaluated here because the EC2 hosts were terminated.

1.3 What is genuinely contributed

The repository's own research pack recommends (and I agree) that the defensible contribution is *not* either named method. It is:

1. the finding that **parallel-aware scaling is the capability that matters**: restricting the hardware term to plan nodes beneath a `Gather` reduces error from 66.97% to 22.23–22.41%;
2. the finding that **cross-ISA transfer fails and must be gated** by calibration or by refusal;
3. a set of **negative results reported with confidence intervals**: the roofline weighting and the per-operator weighting are statistically indistinguishable from a single scalar once parallelism is handled (-0.17 pp, CI $[-0.49, +0.10]$), and a learned model does not beat the analytical formula.

It is designed to be reproducible: canonical numbers are regenerated by one script and the Makefile compares two runs byte-for-byte (the byte-identical outcome is **[PACK]**; I did not re-run it).

1.4 What it is not

- It is not a production-ready predictor: HEAD is an alpha package, with no measured end-to-end accuracy on a real production database. The only "production" evidence is four GitHub-Actions runners and eight local containers.
- It does not predict the plan: it assumes the production planner picks the same plan. On the Docker grid 0 of 22 plans differ across cells, so plan changes are untested there; across PostgreSQL versions 12 of 22 plans change (**[FILE]** `results/tier2_versions.csv`, as reported by the research pack).
- It does not use hardware performance counters; AWS Nitro hides them (Chapter 2).

1.5 System at a glance

Figure 1.1 shows the end-to-end flow; the numbers in each box are the sections that explain it.

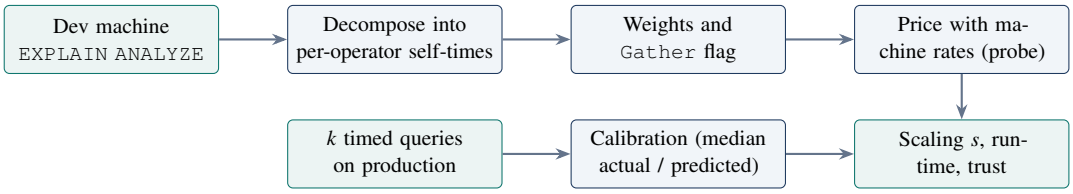


Figure 1.1: Gen-2 (per-operator) prediction path. Derived from `cetp/plan_nodes.py`, `cetp/operator_model.py` and `cetp/cross_env.py`.

1.6 Two generations, one repository

	Gen 1 "roofline"	Gen 2 "per-operator"
Machine signature	2 scalars: STREAM bandwidth, multi-core GFLOPS	Per-operator rates from a PostgreSQL micro-probe; parallel speed-up
Query model	Whole-query: one global factor	Decomposed plan: weights over operators, `Gather` aware
Formula	$\sigma = 0.5/B + 0.25/C + 0.25\sqrt{1/(BC)}$	$\sigma = \sum_b w_b \rho_b \pi_b \kappa$ (Chapter 8)
Evidence	5 EC2 machines, 840 pairs	8 Docker cells (1,232 pairs); 11 laptop-to-prod cells (242 pairs)
Exposed in	CLI, VS Code extension, CI gate	CLI (`predict`, `calibrate`, `check`); not the extension or CI gate

The Gen-1 bandwidth B and compute C are ratios prod/dev. The "roofline" label is historical: the formula is a fixed-weight blend, not the minimum of a compute and a bandwidth bound [21]. A reader who finds "roofline" in the code or older decks should read it as "Gen-1 analytical".

1.7 Reading guide to the numbers

Two result sets are quoted in this repository: the *canonical* numbers (`results/canonical_metrics.json`, regenerated byte-identically by one script) and older "corrected" numbers (`results/corrected_headline_mape.json`). They differ in the second decimal (analytical 21.61 vs 21.599; naive-linear 23.05 vs 23.025). This document uses canonical values and labels older ones. The Review 3 slide deck uses the older values; this is not an error of substance but the two must not be mixed in one table (Appendix I).

2 Problem statement and requirements

2.1 The problem

A developer tunes a query on a laptop or staging server. Production has different CPUs, memory bandwidth, core counts, storage and often a different instruction set. The question "will this meet the latency budget there?" is normally answered by deploying and measuring, which is slow and sometimes dangerous. CETP tries to answer it before deployment, using only information obtainable without privileged access to production.

2.2 Why it is hard

1. **A query is not one thing.** Its runtime mixes sequential scanning (bandwidth-bound), hash building and sorting (cache- and compute-bound), nested loops (latency-bound) and parallel workers. Machines differ on each of these by different factors (serial-axis spread up to 1.39× on the Docker grid versus 3.57× for parallelism, **[FILE]** `results/operator_model_metrics.json`).
2. **Hardware counters are unavailable.** The AWS Nitro hypervisor blocks the PMU, so the well-known "counter-based prediction" route is closed for the target cloud **[CLAIM]**.
3. **Noise is large relative to the signal.** The first repetition of a cell is the slowest in 61 of 105 cells, worst case 8.33×; after discarding it, 11 of 105 cells still exceed 5% CV (**[FILE]** `results/canonical_metrics.json`). Error numbers below about 5% are meaningless.
4. **The planner can change its mind.** Different versions or statistics can choose a different plan; 12 of 22 TPC-H plans change across PostgreSQL 14/15/17/18 (research pack, **[PACK]**).
5. **Only a few machines exist.** Five EC2 types and eight containers are tiny samples of the population of machines, so any learned model is fitted on very few distinct hardware points.

2.3 Scope: what is in and out

In scope	Out of scope (stated or observed)
Predict runtime *scaling* for a query whose dev plan is known	Predict plan changes; the model assumes the same plan
Analyse EXPLAIN output; run a probe that creates and drops its own `UNLOGGED` tables	Touching user data. Note the probe is not read-only: it needs `CREATE` privilege and about 8.4 million scratch rows across three tables (2.0M + 0.4M + 6.0M)
Confidence and refusal when extrapolating (low confidence, cross-ISA)	Absolute-time models of a machine unseen on both sides
Concurrency scaling via a simple power law N^e	Lock contention, buffer-pool eviction, I/O-bound workloads (classifier recall for I/O is 0)
TPC-H scale factor 1 as the benchmark	Real customer workloads (none were measured)

2.4 Requirements

These are reconstructed from the repository (README, CLI, tests, research pack). The original capstone requirement document was not provided; where a requirement is inferred from code rather than a stated document, it is marked. IDs are used by the traceability matrix in Appendix G.

2.4.1 Functional requirements

ID	Requirement	Source of requirement
FR-1	Capture a hardware signature of a machine (bandwidth, compute; operator rates)	README; `scripts/collection/operator_probe.py`
FR-2	Decompose an EXPLAIN ANALYZE plan into per-operator self-times	`cetp/plan_nodes.py`; tests
FR-3	Predict a scaling factor and runtime from dev plan + two signatures	`cetp/cross_env.py`
FR-4	Refuse or warn when extrapolating (cross-ISA uncalibrated, low confidence); **partially met**: the CLI path bypasses the refusal (Section 9.9)	`cetp/cross_env.py`; tests
FR-5	Calibrate from $k \geq 3$ production observations; persist atomically	`cetp/calibration_store.py`
FR-6	Apply a concurrency factor N^e with elasticity e	`cetp/operator_model.py`
FR-7	Gate a query against an SLA with PASS/WARN/BLOCK	`gate/cetp_gate.py`
FR-8	Detect regressions between two plans on the same hardware	`cetp/regress.py`
FR-9	Ingest queries from `auto_explain`/`pg_stat_statements`	`cetp/ingest.py`
FR-10	Redact sensitive literals/identifiers before sharing plans	`cetp/redact.py`
FR-11	Local telemetry without query text or identifiers	`cetp/telemetry.py`
FR-12	Command-line interface for all of the above	`cetp/cli.py`

2.4.2 Non-functional requirements

ID	Requirement	Where checked
NFR-1	Reproducibility: published numbers regenerate byte-for-byte	`make verify-determinism` ([PACK])
NFR-2	No runtime dependencies; Python $\geq$ 3.11	`pyproject.toml`
NFR-3	Leakage-free evaluation (no test-fold information in fitting)	Leakage audit in `canonical_evaluation.py`
NFR-4	Honest uncertainty: report CIs, paired differences, and negative results	`results/CANONICAL.md`
NFR-5	Privacy: telemetry and shared plans contain no query text/identifiers	`cetp/redact.py`, `cetp/telemetry.py`
NFR-6	Crash-safe persistence of calibration	atomic write in `cetp/calibration_store.py`
NFR-7	Probe must not leave artefacts on the target database	`finally`-block drops in `operator_probe.py`

2.4.3 Success criteria

The repository states its own bar implicitly: a predictor is "useful" only if it beats the naive baseline with a confidence interval that excludes zero. By that bar, the status per scenario is:

Scenario	Best model	Naive	Verdict
EC2 LOMO (840 pairs)	21.61	31.96	Met vs naive; not vs compute-only
Docker grid (1,232 pairs)	22.41	72.52	Met, large margin
Laptop $\rightarrow$ prod, same ISA (7 cells)	29.31	37.39	Met
Laptop $\rightarrow$ prod, all 11 cells	43.72	31.70	**Not met**
Laptop $\rightarrow$ prod, x86 runners (4)	68.92	21.74	**Not met** ($3\times$ worse)
Laptop $\rightarrow$ EC2 (95 pairs)	160.68	129.39	**Not met**

All values are MAPE (%), [FILE]; sources in Appendix H.

2.5 The central trade-off

A predictor can be safe by being right more often or by being honest about when it is not. CETP does the second at best: confidence-aware abstention helps modestly: dropping the least confident 20% of predictions leaves a selective MAPE of 28.13%, and the area under the risk–coverage curve is 25.44 for hardware-distance ordering against 31.15 for random ordering ([FILE] `results/confidence_abstention_summary_corrected.json`), but this does not turn an inaccurate predictor into an accurate one.

3 Background and related work

3.1 Concepts a reader needs

3.1.1 How PostgreSQL reports time

`EXPLAIN (ANALYZE, FORMAT JSON)` executes the query and returns a tree of plan nodes. Each node carries `Actual Total Time` (the time to produce all its rows, *per loop*), `Actual Loops`, `Actual Rows`, and buffer counters; the document root carries `Execution Time` [17]. Three details drive the design of CETP.

1. **Times are inclusive.** A node's time contains its children's time. The node's own work (its *self-time*) is its inclusive time minus its children's inclusive time.
2. **Times are per loop.** A node executed L times (for example the inner side of a nested loop) reports the *average* time of one execution, so total inclusive time is `Actual Total Time` $\times L$.
3. **Parallel nodes report averages across participants.** Below a `Gather` node, the leader and workers each run the subtree; the reported per-node time is averaged, so node self-times sum to roughly the total CPU time, not wall time. I measured the ratio on the three real fixtures at 2.97× (q19) and 3.00× (q1) **[RUN]** (Section 9.8); the source comment says 2.85×.

3.1.2 Why parallelism dominates cross-machine differences

PostgreSQL launches up to `max_parallel_workers_per_gather` workers (default 2, giving a degree of parallelism of 3 with the leader). If a container is limited by a CPU quota (the Linux CFS scheduler, `docker --cpus`), three processes time-share fewer cores and the *parallel speed-up* falls below 1: measured 0.769 at one CPU and 2.64–2.70 at two or more (Table 13.1). A scalar capability ratio cannot represent this; it affects only the nodes below a `Gather`.

3.1.3 Metrics

Let y_i be the measured production runtime and $\hat{y}_i$ the prediction. Because $\hat{y}_i = T_i^{\text{dev}}\,\hat{s}_i$ and $y_i = T_i^{\text{dev}}\,s_i$ share the same development time, relative errors on runtime and on the scaling factor s are identical, and the evaluation scripts score s directly.

$$\text{MAPE} = \frac{100}{n}\sum_i \frac{|\hat{y}_i - y_i|}{y_i}, \qquad \text{MdAPE} = 100 \cdot \text{median}_i \frac{|\hat{y}_i - y_i|}{y_i}, \qquad q_i = \max\left(\frac{\hat{y}_i}{y_i}, \frac{y_i}{\hat{y}_i}\right).$$

MAPE is dominated by over-predictions (no upper limit) and is asymmetric; the repository therefore also reports MdAPE and q-error [15]. All differences quoted as significant use a *cluster* bootstrap: whole machines (folds) are resampled with replacement 2,000 times (seed 42), so correlated pairs from the same machine are not counted as independent evidence [4].

3.1.4 Cross-validation protocols

Protocol	What is held out
LOMO	one machine: tests generalisation to unseen hardware (the claim that matters)
LOQO	one query: tests generalisation to unseen workloads
LOMQO	one machine *and* one query: both unseen at once

A leakage-free implementation refits *every* learned component (classifier, quantile models) inside each training fold; the repository audits this explicitly (`results/canonical_leakage_audit.csv`). Leakage is a known source of inflated estimates in applied machine learning [10].

3.2 Related work

CETP sits between three literatures.

Learned query performance prediction. Early work predicts execution time from plan or query features with regression or kernel methods [1, 6]; later work asks whether the optimizer's own cost model is usable after calibration [22]. Plan-structured neural networks [13] and tree-based end-to-end estimators [19] model the plan as a tree. These approaches train and test on the *same* system and hardware. CETP's setting differs: the target *machine* is the unseen variable. The most related line is zero-shot cost models [9], which generalise across *databases* using transferable plan features. CETP instead transfers across *hardware*, with an explicit physical model rather than a learned representation, and with a tiny training set (five or eight machines).

Performance modelling and hardware characterisation. The roofline model [21] bounds throughput by $\min(\text{peak compute}, \text{bandwidth} \times \text{intensity})$. The STREAM triad [14] measures sustainable memory bandwidth. CETP's Gen-1 signature (bandwidth plus multi-core GFLOPS) borrows these two measurements but replaces the minimum with a fixed-weight blend; the repository's own finding is that the weighting does not matter. The Gen-2 probe instead times PostgreSQL's own operators, which sidesteps the argument that a floating-point signature does not describe a database workload.

Cardinality and optimizer evaluation. Leis et al. [11] showed that cardinality errors, not the cost model, dominate plan quality. CETP avoids cardinality estimation by taking measured node times from `EXPLAIN ANALYZE` and, in its implemented-but-unvalidated cardinality term, only scales by a row ratio.

Uncertainty and selective prediction. Split conformal prediction gives distribution-free marginal coverage under exchangeability [3]; quantile regression with gradient boosting [5] gives conditional estimates without that guarantee. The risk–coverage view of abstention follows Geifman and El-Yaniv [7]. CETP uses all three and reports where each fails (Section 13.5).

Measurement methodology. Cloud runtime measurements vary substantially between instances and over time [12, 18], and small environmental differences can change benchmark conclusions [16]. The warm-up discard, repetition counts and CV thresholds used in Chapter 7 follow this concern.

What is, as far as I can tell, new here. I did not find prior work that (i) isolates `Gather`-subtree parallelism as the dominant cross-machine term for PostgreSQL scaling, (ii) reports that a roofline-style signature fails across instruction sets, and (iii) reports the associated negative results with cluster-bootstrap intervals. This is a statement about my limited literature search (one session, six papers verified by search, the rest from memory), not a novelty proof; a thesis-level claim needs a systematic review.

4 System architecture

4.1 Architectural style

CETP is a Python library with a command-line front end, plus an offline research pipeline. It is *not* a service: there is no daemon, no database of its own, and no network listener. State is held in three places, all plain files: the operator-signature registry (`models/operator_signature.json`), the per-pair calibration store (`~/.cetp/calibration.json`), and an append-only telemetry log (JSON Lines). This keeps the product auditable and the runtime dependency set empty (`pyproject.toml`: `dependencies = []`, **[FILE]**).

4.2 Component view

Figure 4.1 groups the code by responsibility. The three horizontal tiers are independent enough to be read alone: acquisition (obtain measurements), modelling (turn measurements into a prediction), and interface (expose it).

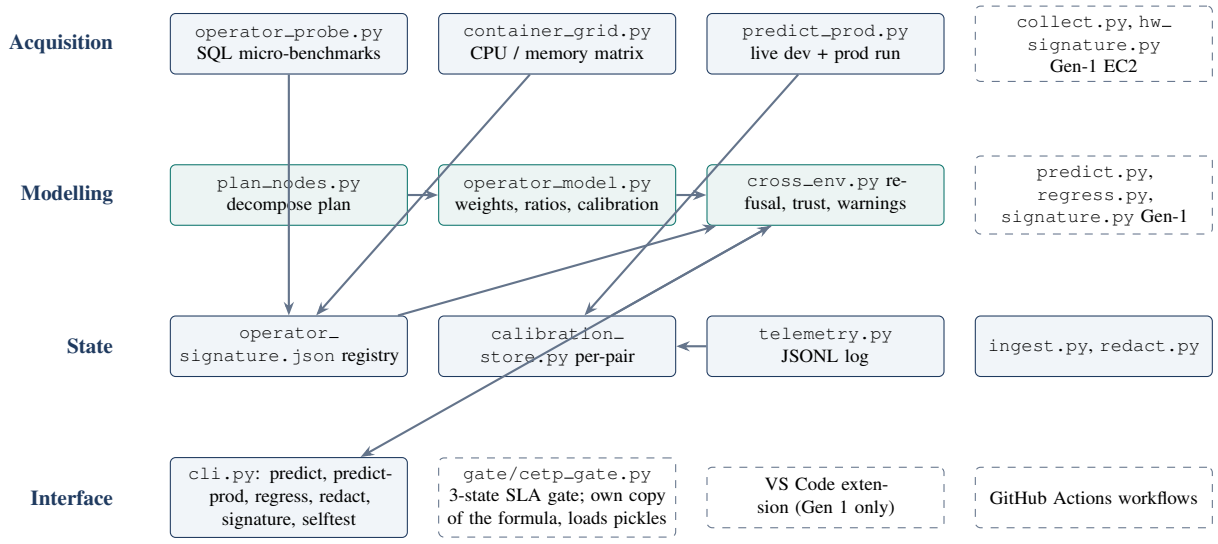


Figure 4.1: Component view. Solid teal = Gen 2 (per-operator); dashed = Gen 1 (two-scalar). Arrows show the main data flow; they are *not* Python imports. Checked against the `import` statements (audit pass): `cross_env` imports `operator_model` and `plan_nodes`; `operator_model` imports `plan_nodes`; `calibration_store` imports `operator_model`; `predict` and `regress` import `signature`; `ingest` imports `redact`; `telemetry`, `redact`, `plan_nodes` and `signature` import nothing from the package; the gate imports none of them.

Honest reading

The figure also shows the most important structural fact about the repository: the user-facing surfaces that exist outside the CLI (the VS Code extension and the two CI workflows that make predictions, `cetp-gate.yml` and `cetp-regression-check.yml`) call Gen-1 code only; the third workflow, `x86-operator-probe.yml`, runs the Gen-2 probe. The newer per-operator model is reachable through `cetp predict-prod` and `scripts/collection/predict_prod.py`. This is documented in the evidence pack (risk R-series) and re-confirmed here by reading `gate/cetp_gate.py`, which loads `models/*.pkl` and calls the bottleneck-weighted formula.

4.3 Runtime data flow for one prediction

Figure 4.2 is the sequence for the Gen-2 path, from CLI invocation to printed result.

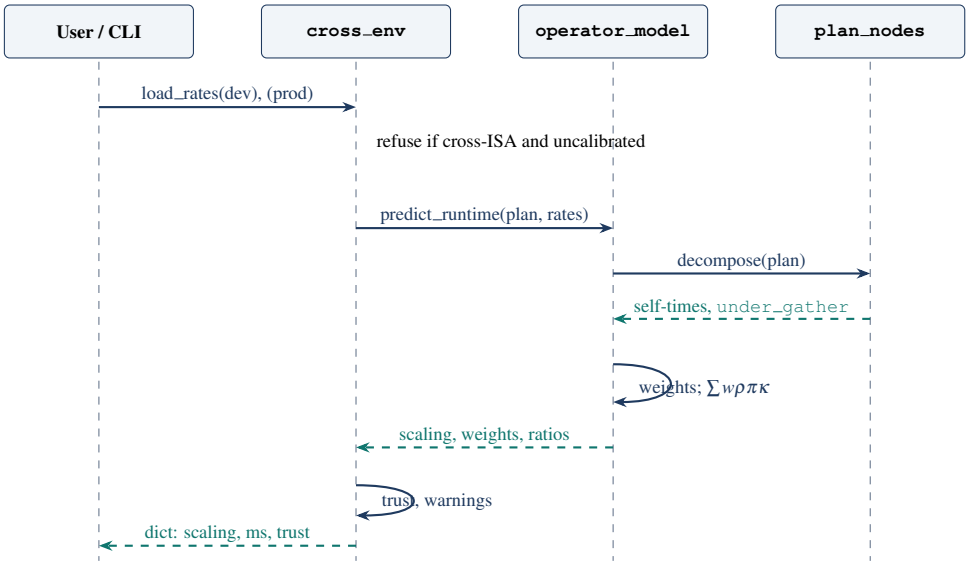


Figure 4.2: Sequence for one Gen-2 prediction. The cross-architecture refusal fires inside `cross_env.predict` only if the caller passed the machine records (see Section 9.9).

4.4 Deployment topology

There is no deployed service. The realistic topologies are:

1. **Local developer.** Installs the package (the exact command, e.g. `pip install -e .`, is inferred from `pyproject.toml`, not verified [UNK]); the developer runs `cetp predict-prod` with two EXPLAIN JSON files and two registry identifiers.
2. **Live harness.** `scripts/collection/predict_prod.py` connects to dev and prod PostgreSQL through `psycopg2`, runs `EXPLAIN` on production (no `ANALYZE`), forces that plan on dev using `pg_hint_plan`, measures, predicts, and records into the calibration store. It needs a database, so I did not run it ([UNK]).
3. **CI.** The Gen-1 gate workflow (`.github/workflows/cetp-gate.yml`) installs the pinned requirements and runs `gate/cetp_gate.py` against a stored SLA configuration; a BLOCK verdict is deliberately *not* a build failure (commit `ffce2ec`).
4. **Reproducibility artefact.** A `python:3.13-slim` Docker image reruns the analysis from committed data with no database (`Dockerfile`).

4.5 Design principles visible in the code

1. **Refuse rather than mislead.** Several functions raise on unsafe inputs: v1 signatures, unreliable probes, mismatched probe scales, and uncalibrated cross-ISA prediction.
2. **Every prediction carries its own critique.** The result dictionary always contains `explained_frac`, `unmapped_frac`, a trust level and a list of warnings.
3. **Zero dependencies in the shipped path.** The pickled scikit-learn models are confined to the Gen-1 gate and the research scripts, because pickles are version-coupled.
4. **Provenance is a first-class artefact.** `models/PROVENANCE.json` records library versions and a `version_mismatch: false` flag, and result generation is hash-checked (Section 17.3).

5 Technology stack

5.1 Languages and runtimes

Component	Version	Role and why	Evidence
Python	≥3.11 declared; CI and Docker use 3.13	All logic. Declared in `pyproject.toml`; 3.13 in `Dockerfile`. The selftest also passed on 3.10.12 here, below the declared floor, because the shipped package uses only the standard library	**[FILE]**; **[RUN]**
PostgreSQL	15 (probe/grid), 14/15/17/18 (plan studies)	Target system. `postgres:15` image in `container_grid.py`; four versions in `data/plans_pg*`	**[CODE]**
`pg_hint_plan`	not pinned	Force production's plan on the dev server so plan divergence is removed by construction	**[CLAIM]** (module docstring)
C	C11	Gen-1 hardware probes (`hw_probe_suite.c`, `compute_probe.c`, `membw_antagonist.c`). Not read in full	**[UNK]**
TypeScript / Node	not recorded here	VS Code extension `cetp-vscode-0.2.4.vsix` (Gen 1 only)	**[FILE]**
GNU Make	any	Reproduction entry points (Chapter 14)	**[FILE]**
Docker	any	Container grid (CPU/memory matrix) and artefact image	**[FILE]**

5.2 Libraries and pinned versions

Pinned exactly in `requirements.txt` (**[FILE]**). The pin is not decorative: unpinned scikit-learn made the Gen-1 gate fail with `No module named '_loss'` when unpickling the quantile models (comment in `requirements.txt`).

Package	Version	Used for
scikit-learn	1.8.0	Random-forest bottleneck classifier, gradient-boosted quantile models (research and Gen-1 gate only)
numpy	2.5.3	Numerics in evaluation and gate
pandas	2.2.3	Dataset handling in `canonical_evaluation.py`
scipy	1.18.1	Statistics (Pearson *r*, bootstrap helpers)
joblib	1.5.3	Model (de)serialisation
PyYAML	6.0.2	Gate SLA config files
matplotlib	3.10.1	Figures in analysis scripts
pytest	9.0.3	Test runner
psycopg2-binary	≥2.9	Live PostgreSQL access (`research` extra only)
setuptools	≥68	Build backend

The version numbers above are as written in the repository; I did not independently check that each released on the

stated date (`numpy 2.5.3` and `scipy 1.18.1` are after my own knowledge of releases, so I treat them as the repository's record, not mine).

5.3 Choices a reviewer will ask about

Choice	Reason given or inferable	Cost
Zero runtime dependencies	Avoid version coupling; ship formula + JSON table instead of a pickle (`pyproject.toml` comment)	The shipped package cannot use the learned models
JSON / JSONL files, no database	Auditable, diff-able, atomic write via `os.replace`	No concurrent writers; no query capability
SQL-only probing (Gen 2)	Works on managed PostgreSQL where there is no shell (`cross_env.py` docstring)	Measures PostgreSQL's operators, not the raw hardware
Docker CPU quotas to create "machines"	Cheap, controllable variation in parallelism	Quota is not the same as different silicon: all 8 cells run on one host and one ISA
GitHub Actions x86 runners	Free real x86 targets for the cross-ISA test	Shared, noisy VMs; four near-identical machines (probe spread 2.160–2.167)

5.4 Dependency risks

- **Pickle coupling** between scikit-learn versions affects only the Gen-1 gate and research scripts. `models/PROVENANCE.json` states the pickles were rebuilt under 1.8.0.
- **PostgreSQL version.** Plans change between 14/15/17/18 (12 of 22 TPC-H plans), and the decomposer's operator map was extended after a corpus sweep found `BitmapAnd` unmapped (`plan_nodes.py` lines 46–52). A future PostgreSQL release introducing a new node type would silently land in the unmapped residual, which is reported but not an error.
- **`pg_hint_plan` silently ignores hints it cannot express**, which is why the code carries `plan_forced=False` and a LOW trust flag (`cross_env.py` lines 26–28, 191–192).

6 Repository walkthrough

The repository root is `CETP/`. Line counts are from `wc -l` on the checked-out tree **[RUN]**. The tree is large because it contains both the product and the evidence behind it.

6.1 Top-level layout

Path	Purpose	Size / state
`cetp/`	The installable package: 11 modules + `__init__.py` (empty)	2,278 lines
`gate/`	Gen-1 SLA gate: `cetp_gate.py` (326), `cetp_predict_cli.py` (152), `cetp_gate_demo.py` (88)	uses pickles
`scripts/collectior`	Data acquisition: probes, container grid, TPC-H/DS/JOB loaders, live predictor, C sources	35 entries (21 Python, 3,336 lines)
`scripts/analysis/`	Evaluation: one canonical script plus 33 experiment scripts (tiers 1–4, phases 5–8, additions, fixes) and one notebook	34 Python files, 8,852 lines
`scripts/docs/`	PDF build tooling for the documentation (the Markdown it consumes was deleted in HEAD)	broken
`tests/`	4 test files + 3 real EXPLAIN fixtures (`q1`, `q18`, `q19`)	1,260 lines
`models/`	Registry JSON, `calibration.json`, 4 pickles, `PROVENANCE.json`, gate config	2.1 MB
`data/`	Measurements: `corrected/`, `container_grid/`, `laptop/`, `plans_*`, `external/` (23 MB)	mixed
`results/`	78 committed outputs (JSON/CSV/PNG); `CANONICAL.md` is the source of truth	1.4 MB
`docs/`	Evidence pack (`research/00-09`), paper draft (`paper/CETP_paper.tex`), PDF reports, static site	14 MB
`vscode-extension/`	VS Code panel (v0.2.4, Gen 1 only)	built
`.github/workflows/`	`cetp-gate.yml`, `cetp-regression-check.yml`, `x86-operator-probe.yml`	3 workflows
`dist/`	`cetp-datasets.tar.gz` and `MANIFEST.json` (dataset package)	artefact
`models_backup/`, `models_online/`	Earlier pickles; online-learning model versions	legacy
`Makefile`, `Dockerfile`, `pyproject.toml`, `requirements.txt`	Build, reproduce, package	–

Note

Git state at the time of writing (**[RUN]**): branch `x86-probe`, HEAD `38c4bf2` "Delete all markdown except README; point README at the PDF docs", 17 commits (first: `366c5d4` "Version 1"), working tree modified (`M .gitignore`, `M Makefile`, `M README.md`, `M vscode-extension/*`, regenerated Gen-2 results) with a large uncommitted documentation reorganisation: the PDFs at their old `docs/` paths are deleted, re-appear untracked under `docs/{design,guides,reference,results}/`, and `docs/research/*.md`, `docs/README.md`, `results/CANONICAL.md` and `tests/test_operator_evaluation.py` are untracked. Fourteen of the 17 commits are dated 2026-10-13, the first three 2026-08-17. Several things this document relies on (the C10 fix, the regenerated Gen-2 results, `docs/paper/CETP_paper.tex`) are *uncommitted*.

6.2 The package: module by module

Module	Lines	Responsibility and key symbols
`plan_nodes.py`	192	Decompose an EXPLAIN plan: `OPERATOR_MAP` (l. 34), `AGG_STRATEGY` (56), `PASSTHROUGH` (60), `COMPLEXITY` (68), `operator_for` (77), `decompose` (98), `operator_profile` (179)
`operator_model.py`	261	The Gen-2 model: `AXES` (41), `DEFAULT_CONCURRENCY_ELASTICITY=0.734` (51), `cardinality_factor` (68), `weights_from_plan` (89), `concurrency_factor` (132), `predict_scaling` (140), `predict_runtime` (207), `fit_calibration` (221), `select_calibration_queries` (239)
`cross_env.py`	264	Product flow: `CrossArchitectureUncalibrated` (63), `load_rates` (84), `assert_comparable` (112), `row_ratio_from_plans` (120), `predict` (141), `_trust` (184), `_warnings` (205)
`calibration_store.py`	168	Per-pair, per-query calibration state; `MIN_OBSERVATIONS=3` (43); atomic save (101)
`telemetry.py`	199	Local JSONL log; `_FORBIDDEN` keys (46); record/resolve/summary
`ingest.py`	202	Parse `auto_explain` logs (105), rank `pg_stat_statements` (161, 189)
`redact.py`	218	Structural/full redaction, `assert_clean` (145), CLI `cetp-redact` (178)
`regress.py`	103	Regression detection on same-hardware plan pairs; threshold 1.20 (30)
`predict.py`	154	Gen-1 predictor: `analytical_scaling` (67), `compute_only_scaling` (82), `hardware_distance_confidence` (88), `predict` (107)
`signature.py`	141	Gen-1 machine table `LEGACY` (31), registry loading, ratios (105), warnings (119)
`cli.py`	376	Entry points: `cmd_predict` (64), `cmd_regress` (96), `cmd_redact` (112), `cmd_signature` (117), `cmd_predict_prod` (146), `cmd_selftest` (220), parser (293)

6.3 Where the evidence lives

Question	File
Headline numbers, CIs, ablations	`results/CANONICAL.md`, `results/canonical_metrics.json`, generated by `scripts/analysis/canonical_evaluation.py` (854 lines)
Gen-2 container-grid evaluation	`results/operator_model_metrics.json`, `results/operator_model_pairs.csv`, from `operator_model_evaluation.py` (518 lines)
Laptop → production	`results/laptop_to_prod_metrics.json`, `laptop_to_prod_pairs.csv`
Laptop → EC2	`results/laptop_to_ec2_metrics.json`, `laptop_to_ec2_pairs.csv`
Gate behaviour	`results/phase7_gate_evaluation.csv`, `tier1_gate.csv`
Measurement noise	`results/canonical_measurement_cv.csv`
Leakage audit	`results/canonical_leakage_audit.csv`
Conformal calibration	`results/tier1_conformal.csv`, `models/calibration.json`
Gaps, contradictions, risks	`docs/research/06_gaps_and_traceability.md`, `08_method_comparison_and_contribution.md`, `09_C10_investigation_and_fix.md`

6.4 Reading order for a new contributor

1. `README.md`, then `results/CANONICAL.md` (one hour).
2. `cetp/plan_nodes.py` then `cetp/operator_model.py`: the whole Gen-2 idea is about 450 lines.

3. `tests/test_operator_model.py`: it is the best executable specification of the model.
4. `cetp/cross_env.py` for refusals and trust.
5. `scripts/analysis/canonical_evaluation.py` `run_cv` (line 239): how folds and leakage are handled.

6.5 Hazards in the tree

- `data/raw_buggy_do_not_use/` holds the original, buffer-inflated dataset; the name is the warning (the bug summed buffer counters across the tree, inflating them 3.06–12.00×).
- `data/plans_grid` (1.1 MB) is untracked and git-ignored, so a fresh clone cannot rerun the per-operator evaluation (**[PACK]**, risk list).
- `cetp_measurements_addition7_demo.db` at the root is a SQLite demo database from the online-learning experiment.
- `scripts/docs/` cannot rebuild most of the documentation: it renders Markdown to PDF, and in HEAD only `README.md` survives (the working tree adds untracked `docs/README.md`, `docs/research/*.md` and `results/CANONICAL.md`).

7 Detailed implementation

This chapter documents the functions that decide whether a prediction is right or wrong. Each card has the same nine items: purpose, inputs, logic, outputs, dependencies, edge cases, failure modes, verification and location. Line numbers are those of the file at HEAD plus working-tree changes (**[CODE]**). Test names refer to `tests/`; "**[RUN]**" means I executed it.

7.1 Plan decomposition

decompose(plan) — cetp/plan_nodes.py:98–176

Purpose. Turn an `EXPLAIN (ANALYZE, FORMAT JSON)` tree into a flat list of nodes with *self*-time, correcting for per-loop averaging, and tag each node with whether it lies below a `Gather`.

Inputs. A plan in any of three shapes: the full EXPLAIN list `[{"Plan": ...}]`, a dict with `"Plan"`, or a bare node (`_unwrap`, l. 87).

Logic. Depth-first walk. For node n the inclusive time is $I_n = \texttt{Actual Total Time} \times \texttt{Actual Loops}$ (l. 113–118). Self-time is $S_n = \max\left(0,\ I_n - \sum_{c \in \text{children}} I_c\right)$ (l. 131–134). `under_gather` is inherited downward and becomes true after passing a `Gather` or `Gather Merge` (l. 155–157). `parallel` is set if any node is `Parallel Aware` or a Gather. The operator axis comes from `operator_for`.

Outputs. Dict with `nodes`, `execution_time_ms`, `parallel`, `analyzed`, `self_time_sum_ms`, `residual_ms`, `residual_frac`. Each node: `node_type`, `operator`, `self_ms`, `inclusive_ms`, `rows` (rows × loops), `plan_rows`, `loops`, `depth`, `parallel_aware`, `under_gather`.

Dependencies. None beyond the Python standard library; `operator_for`, tables `OPERATOR_MAP`, `AGG_STRATEGY`, `PASSTHROUGH`.

Edge cases. (i) Plans without ANALYZE: `Actual Total Time` is absent, `self_ms=None`, and `analyzed` is false. (ii) Parallel averaging can make the subtraction negative; it is clamped to 0 (l. 132–134), which silently discards time. (iii) `Actual Loops` of 0 or missing is treated as 1 (`or 1`). (iv) Plans from PostgreSQL 18 with extra fields do not break parsing (`test_pg18_only_fields_do_not_break_decomposition`).

Failure modes. Empty list raises `ValueError("empty plan")`. A new node type is not an error: it gets `operator=None` and its time falls in the unmapped residual. Self-times that do not sum to `Execution Time` produce a large `residual_frac`; the function reports but does not reject.

Verification. `test_self_time_subtracts_children`, `test_per_loop_time_is_multiplied_by_loops`, `test_negative_self_time_is_clamped`, `test_under_gather_not_parallel_aware`. **[RUN]** I ran it on the three real fixtures: q1 has `Seq Scan` self-time 7,533 ms and `Aggregate` (hashed) 3,214.5 ms; node self-times sum to 3.00× the wall time (Section 9.8).

Location. `cetp/plan_nodes.py`, lines 98–176.

operator_for(node) and the operator map — cetp/plan_nodes.py:34–84

Purpose. Decide which probe axis prices a node.

Inputs. A plan node dict (`Node Type`, and `Strategy` for aggregates).

Logic. `Aggregate` → by strategy: Hashed/Mixed → `hashagg`, Sorted → `sort`, Plain → `seqscan`. Nodes in `PASSTHROUGH` (Gather, Limit, Result, Append, Materialize, Memoize, WindowAgg, ...) → `None`. Otherwise `OPERATOR_MAP`: Seq Scan → seqscan; Hash/Hash Join → hashjoin; Sort/Incremental Sort/Merge Join → sort; Nested Loop and *all* index and bitmap scans → nestloop.

Outputs. One of `seqscan`, `hashjoin`, `sort`, `hashagg`, `nestloop`, or `None`.

Dependencies. Constants only.

Edge cases. Missing `Strategy` defaults to "Plain" → seqscan. Unknown strategy falls to `hashagg`. `BitmapAnd/Or` were added after a corpus sweep over PostgreSQL 14/15/17/18 found them unmapped.

Failure modes. The mapping is a modelling assumption, not a measurement: treating Index Scan as `nestloop` is justified only by the comment that the probe's nestloop axis is an index-driven loop. A workload dominated by index scans is priced by an axis that was never validated against it.

Verification. Parametrised `test_operator_mapping`; `test_aggregate_splits_by_strategy`; `test_bitmap_combination_nodes_have_an_axis`.

Location. `cetp/plan_nodes.py`, lines 34–84.

7.2 The per-operator model

weights_from_plan(plan, dev_speedup) cetp/operator_model.py:89–129

Purpose. Compute the normalised *wall-time* share of each (axis, parallel) bucket, plus the unmapped share.

Inputs. Plan; `dev_speedup` π_d (default 1.0), the development machine's measured parallel speed-up.

Logic. For each node with $S_n > 0$: $\text{par}_n = \texttt{under_gather}_n$ (not `Parallel Aware`). Parallel nodes are converted from CPU-time to wall-time by $t = S_n/\pi_d$. Mapped time accumulates in bucket (op, par) and unmapped time in a residual. Weights are $w_b = t_b / \left(\sum_b t_b + t_{\text{unmapped}}\right)$; the largest row count per bucket is kept for the cardinality term.

Outputs. Tuple `(weights, unmapped_frac, profile)`; `weights` sums to $1 - \texttt{unmapped_frac}$.

Dependencies. `decompose`.

Edge cases. If total time is 0 returns `({}, 1.0, ...)`, meaning "everything unmapped". `dev_speedup` of 0/None skips the conversion.

Failure modes. If the dev speed-up is mis-measured (the dev machine was contended), every parallel weight is wrong by the same factor; the normalisation hides that but shifts mass between serial and parallel buckets. Keying on `Parallel Aware` instead corrected only 46% of q19's time and predicted 1.49× against a true 3.01× (code comment, l. 145–152).

Verification. `test_weights_sum_to_one`, `test_unmapped_time_is_reported_not_dropped`, `test_parallel_speedup_changes_the_prediction`.

Location. `cetp/operator_model.py`, lines 89–129.

predict_scaling(plan, dev_rates, prod_rates, ...) cetp/operator_model.py:140–204

Purpose. The model: predict $s = T_{\text{prod}}/T_{\text{dev}}$ and report how much of the plan it could explain.

Inputs. Plan; two rate dictionaries {axis: tuples/s}; optional `row_ratio`, `calibration`, `dev_speedup`, `prod_speedup`, `prod_clients`, `concurrency_elasticity`.

Logic. For each bucket $b = (\text{op}, \text{par})$: $\rho_b = \dfrac{r_d(\text{op})\,\pi_d^{[\text{par}]}}{r_p(\text{op})\,\pi_p^{[\text{par}]}}$ where $\pi^{[\text{par}]}$ equals the speed-up for parallel buckets and 1 otherwise; κ_b is the cardinality factor. Contribution $w_b \rho_b \kappa_b$. Axes lacking a rate on either machine are *unpriced*. The fallback $g = \text{geomean}(\rho_b)$ prices both unmapped share and unpriced weight: $s_0 = \sum_b w_b \rho_b \kappa_b + (u+m)\, g\, \kappa_{\text{none}}$. Final $s = s_0 \cdot c \cdot N^e$ with calibration c.

Outputs. Dict with `scaling`, `scaling_uncalibrated`, `concurrency_factor`, `weights`, `weights_by_parallelism`, `per_axis_ratio`, `unmapped_frac`, `unpriced_axes`, `unpriced_weight`, `explained_frac`, `residual_frac`, `parallel`, plus echo fields (`calibration`, `dev_speedup`, `prod_speedup`, `prod_clients`, `concurrency_elasticity_used`, `concurrency_elasticity_is_default`, `row_ratio`); `predict_runtime` adds `dev_runtime_ms`, `predicted_prod_ms`.

Dependencies. `weights_from_plan`, `cardinality_factor`, `concurrency_factor`.

Edge cases. No priced bucket: $g = 1$. `row_ratio=None`: all $\kappa = 1$. Clients ≤ 1: no concurrency.

Failure modes. (i) The formula predicts a *ratio*, so it needs a measured T_{dev}. (ii) The geometric-mean fallback hides the fact that unpriced nodes may behave very differently from priced ones; the caller must read `explained_frac`. (iii) The model is multiplicative in independent factors, so errors in the serial and parallel speed-ups compound. (iv) Zero or negative rates would raise `ZeroDivisionError` or yield nonsense; `load_rates` does not validate positivity.

Verification. All tests in `tests/test_operator_model.py` (34 definitions, 42 collected); **[RUN]** 80 tests across that file and `test_product_loop.py` passed. Worked numerically in Section 9.8.

Location. `cetp/operator_model.py`, lines 140–204.

cardinality_factor(operator, row_ratio, n_rows) cetp/operator_model.py:68–86

Purpose. Scale a node's cost when production holds a different amount of data.

Inputs. Axis name, row ratio $r = \text{rows}_p/\text{rows}_d$, optional row count n of the node.

Logic. Linear axes: r. Sort (n log n): $r\left(1 + \ln r / \ln n\right)$, the exact ratio of $n \log n$ costs; n defaults to 10^5.

Outputs. A multiplicative factor (1.0 if r is missing or non-positive).

Dependencies. `COMPLEXITY`, `math`.

Edge cases. $n \leq 1$ falls back to the default; $r = 1$ gives exactly 1.

Failure modes. **Unvalidated against data.** Every grid cell holds identical SF1 data, so only $r = 1$ was ever exercised. The CLI keeps it off unless `-use-row-ratio` is passed, and a warning is emitted if $|r - 1| > 0.05$.

Verification. Unit tests on identity, superlinearity and insensitivity to assumed n only (no empirical validation).

Location. `cetp/operator_model.py`, lines 68–86.

fit_calibration(observations) and select_calibration_queries(candidates, k) cetp/operator_model.py:221–261

Purpose. Estimate a per-machine multiplicative correction from k measured queries; choose which k to measure.

Inputs. List of (predicted, actual) scalings; or {query: predicted scaling} and k.

Logic. $c = \text{median}_i(a_i/p_i)$ over valid pairs. Selection: sort by predicted scaling; $k = 1 \Rightarrow$ the median query; else choose queries nearest to k evenly spaced targets between min and max.

Outputs. A float c (1.0 if no valid observation); a list of query ids.

Dependencies. None.

Edge cases. Pairs with non-positive values are dropped; $k \geq$ number of candidates returns all, sorted.

Failure modes. Median of one value is that value, so $k = 1$ with a bad ratio is harmful; this is why the store flags $k < 3$ as provisional (but still applies the factor; see `CalibrationStore`). A single multiplicative factor cannot correct an error that depends on the operator mix.

Verification. `test_calibration_uses_median_not_mean`, `test_calibration_of_nothing_is_identity`, `test_k1_selection_picks_the_median_query`, `test_selection_spreads_over_the_range`, `test_selection_never_repeats_a_query`. Empirical: few-shot MAPE on the Docker grid 22.41 (k=0), 25.26 (k=1), 21.30 (k=2), 21.81, 21.70, 21.02 (k=8) **[FILE]**. Note k=1 is *worse* than k=0.

Location. `cetp/operator_model.py`, lines 221–261.

concurrency_factor(clients, elasticity) cetp/operator_model.py:132–137

Purpose. Scale runtime for N simultaneous clients: $T(N) = T(1)N^e$.

Inputs. Client count, optional elasticity (default 0.734).

Logic. Returns (N^e, e) for $N > 1$, else $(1.0, \text{None})$.

Outputs. Tuple.

Dependencies. `DEFAULT_CONCURRENCY_ELASTICITY`.

Edge cases. Negative or zero clients → factor 1.

Failure modes. e varies 0.286–0.984 across queries and is *not* predictable from the plan (leave-one-query-out MAE 0.191 using plan features versus 0.167 using the global median). The default reduces 8-client error from 73.8% (ignoring concurrency) to 41.8%, still large. Measured on one 8-core laptop only.

Verification. `test_concurrency_factor_is_applied_and_flagged_as_default`, `test_concurrency_at_one_client_is_identity`; data in `results/concurrency_model_metrics.json`.

Location. `cetp/operator_model.py`, lines 132–137.

7.3 Product flow and guards

cross_env.predict(...) cetp/cross_env.py:141–181

Purpose. One cross-environment prediction, with refusal, trust and warnings attached.

Inputs. Production plan (no ANALYZE) or None; dev analysed plan; dev runtime; rates; `plan_forced`; calibration; speed-ups; clients; `dev_rec, prod_rec` (signature records); `allow_uncalibrated_cross_arch`.

Logic. (1) `architectures_differ(dev_rec, prod_rec)` compares the last token of `client_platform`. (2) If they differ *and* `calibration == 1.0` and no override: raise `CrossArchitectureUncalibrated`. (3) A row ratio only if `use_row_ratio=True` (default False); production contributes *plan* rows and development *actual* rows, so it is only as good as production's estimates. (4) `predict_runtime`. (5) Attach architecture fields, plan source, `_trust`, `_warnings`.

Outputs. The model dict plus `cross_architecture, dev_arch, prod_arch, plan_forced, plan_source, trust, warnings`.

Dependencies. `operator_model.predict_runtime`, `plan_nodes.decompose`.

Edge cases. Missing `client_platform` on either side ⇒ architectures treated as equal (`test_missing_platform_info_does_not_trigger_refusal` makes this deliberate). Calibration of exactly 1.0 is the only "uncalibrated" signal, so passing 1.0000001 defeats the guard.

Failure modes. **The CLI never passes `dev_rec/prod_rec`** (Section 9.9), so the guard is dead on that path. The warnings text quotes stale figures (28.54% vs 38.22%, "132 pairs") that do not match the committed metrics (29.31% vs 37.39% same-ISA, 242 pairs).

Verification. `test_cross_arch_uncalibrated_is_refused`, `_with_calibration_is_allowed_but_warned`, `_override_is_possible_but_low_trust`, `test_same_arch_uncalibrated_is_not_refused`. **[RUN]** library call raised the exception for m1-host → gha-x86; the same inputs via the CLI returned a prediction.

Location. `cetp/cross_env.py`, lines 141–181.

load_rates(registry_path, machine_id) cetp/cross_env.py:84–109

Purpose. Return a machine's operator rates, refusing signatures that cannot be trusted.
Inputs. Registry path and machine id.
Logic. Raise `KeyError` if absent; `ValueError` if `signature_kind` is not `operator_rates_v1/v2`, if it is `v1` (no parallel axis), if `invalid_probes` is non-empty, or if `reliable` is `False`. Returns `(rates, record)`.
Outputs. Rates dict {axis: tuples/s}, full record.
Dependencies. `json`.
Edge cases. A missing `reliable` field is tolerated (treated as reliable).
Failure modes. **[RUN]** in the committed registry `c2m1g` is flagged unreliable (probe noise), so it cannot be used as a reference; evaluation still includes its cell, flagged "unreliable" (CV 0.307).
Verification. `test_v1_signature_is_refused`, `test_invalid_probes_are_refused`, `test_unreliable_signature_is_refused`, `test_missing_reliable_field_is_tolerated`, `test_unknown_machine_is_refused`.
Location. `cetp/cross_env.py`, lines 84–109.

_trust(out) and _warnings(out) cetp/cross_env.py:184–259

Purpose. Assign a coarse trust level and list every reason to doubt the prediction.
Inputs. The model output dict.
Logic. Order matters: cross-ISA → LOW if uncalibrated else MEDIUM; `explained_frac`<0.5 →LOW; forcing failed → LOW; uncalibrated → MEDIUM; no production plan → MEDIUM; unmapped share >0.25 → MEDIUM; else HIGH.
Outputs. String LOW/MEDIUM/HIGH; list of warning strings.
Dependencies. None.
Edge cases. HIGH requires: same ISA, calibrated, production plan forced and verified, mapped share high.
Failure modes. The thresholds (0.5, 0.25, 0.1) are hand-set, not fitted; trust is a heuristic, not a calibrated probability. No experiment shows that HIGH-trust predictions are more accurate than MEDIUM ones.
Verification. `test_unforced_plan_is_never_high_trust`, `test_uncalibrated_is_never_high_trust`, `test_forced_plan_can_reach_high_trust`.
Location. `cetp/cross_env.py`, lines 184–259.

CalibrationStore.record / save / calibration cetp/calibration_store.py:54–139

Purpose. Persist measured (predicted, actual) scalings per (dev, prod) pair and expose the median correction.
Inputs. dev id, prod id, query id, predicted and actual scaling (positive), optional architectures.
Logic. Observations are keyed by query id, so re-measuring supersedes. `save` writes a temp file in the target directory then `os.replace` (atomic on POSIX). `calibration` returns {factor, n, provisional, source}; `provisional = n < 3`.
Outputs. JSON on disk at `~/.cetp/calibration.json` or `$CETP_CALIBRATION_PATH`.
Dependencies. `fit_calibration`, `tempfile`, `os`.
Edge cases. A corrupt file is ignored and the store degrades to zero-shot (no exception).
Failure modes. No file locking: two processes can lose each other's updates. The factor is still returned when provisional; applying it is the caller's decision. A user-facing CLI command to read the store is not part of `cli.py`.
Verification. `test_store_round_trips_through_disk`, `test_corrupt_store_degrades_to_zero_shot_rather_than_crashing`, `test_remeasuring_a_query_supersedes_its_old_value`, `test_non_positive_observations_are_rejected`.
Location. `cetp/calibration_store.py`, lines 54–139.

7.4 Privacy, ingestion and telemetry

redact_plan(obj, level, salt) and assert_clean(plan) cetp/redact.py:87–165

Purpose. Remove literal values (and optionally identifiers) from an EXPLAIN document before it leaves the machine, then verify.
Inputs. Parsed plan (list/dict), `level` in {structural, full}, hashing salt.

Logic. Recursive copy. Fields in `EXPRESSION_FIELDS` (Filter, Index Cond, Hash Cond, Output, ...) have string literals replaced by `'?'` and numeric literals by `?`. `Query Text` gets the same, or is replaced by "[redacted]" at `full`. At `full`, identifier fields and identifier-like tokens inside expressions are replaced by `id_` plus a 5-byte BLAKE2s hash of "salt:name". `assert_clean` re-scans for surviving quotes or digits.

Outputs. A redacted copy; `True` or `ValueError`.

Dependencies. `re`, `hashlib`.

Edge cases. Structure fields (`Node Type`, `Strategy`, ...) are never touched.

Failure modes. **[RUN]** (a) PostgreSQL escape-string literals with an escaped quote: `E'O\'Brien secret'` becomes `E'?'Brien secret'`, leaking `Brien secret`, and `assert_clean` still returns True. (b) Dollar-quoted literals `$$secret$$` pass through untouched and `assert_clean` returns True. (c) The default salt `"cetp"` is public and 40-bit hashes of common column names are dictionary-recoverable, so `full` is pseudonymisation, not anonymisation. (d) Structural level keeps relation and column names by design.

Verification. `test_ingested_plans_are_redacted_before_being_returned`, `test_query_text_literals_are_stripped`, `test_query_text_is_dropped_entirely_at_full_level`. The failure cases above are *not* covered by any test.

Location. `cetp/redact.py`, lines 87–165.

parse_auto_explain_log(text, level, strict) and rank_queries(rows) cetp/ingest.py:105–186

Purpose. Extract plans and durations from a PostgreSQL log written by `auto_explain` (JSON format), redact them, and rank queries from `pg_stat_statements`.

Inputs. Log text; level; strict flag. Rows with `total_ms`.

Logic. Regex finds "duration: X ms plan:", then a balanced-brace scan extracts the JSON; if parsing fails, repeated log prefixes are stripped and parsing is retried. Each plan is redacted then verified. Ranking sorts by total time and marks the top 3 as `calibration_candidate`.

Outputs. `(entries, dropped)` where entries are `{duration_ms, plan}`.

Dependencies. `redact`.

Edge cases. JSON that is complete but unparseable is dropped and recorded in `dropped`. An entry whose JSON is *truncated* (no closing brace) is skipped *silently*: `_balanced_json` returns `None` and the loop `continue`s without a record **[CODE]**.

Failure modes. The docstring promises a list of `{duration_ms, plan, query_id}`; the code returns a tuple `(entries, dropped)` and no `query_id`. The statement says there is "no flag to disable" redaction, yet `strict=False` silently drops entries that fail the check. `_balanced_json` is string-aware (it tracks quotes and escapes), so braces inside JSON strings are handled; hostile or interleaved log content was not tested.

Verification. `test_auto_explain_parses_every_log_layout`, `test_unparseable_entry_is_dropped_not_raised`, `test_ranking_is_by_total_time_not_mean`, `test_ranking_marks_a_calibration_budget`.

Location. `cetp/ingest.py`, lines 105–186.

7.5 Regression detection

regress.detect(before, after, dev_sig, prod_sig, ...) cetp/regress.py:30–82

Purpose. Decide whether a code or schema change makes a query slower in production, using a ratio in which the hardware term cancels.

Inputs. Two EXPLAIN ANALYZE plans of the same query on the same dev machine; Gen-1 signatures; threshold (default 1.20).

Logic. Predict both plans. dev_ratio $= T_a/T_b$; prod_ratio $= \hat{T}^a_{\text{prod}}/\hat{T}^b_{\text{prod}} =$ dev_ratio $\cdot\, s_a/s_b$; hardware term $= s_a/s_b$. Verdict REGRESSION if prod ratio ≥ 1.2, IMPROVEMENT if $\leq 1/1.2$, else NEUTRAL. Trust: HIGH if $|1 - \text{hw term}| < 0.02$, MODERATE if < 0.15, else LOW.

Outputs. Dict with verdict, ratios, hardware term, trust sentence, before/after details.

Dependencies. Gen-1 `predict`, `signature`.

Edge cases. Identical plans give ratio 1 and "cancelled".

Failure modes. The Gen-1 scaling s is computed from the two machine signatures alone (the default analytical weights are uniform and the plan is used only to read the dev runtime), so $s_a = s_b$ and the hardware term is *exactly* 1.0 for any pair of plans (for both predictors, since `predict.predict` computes the scaling from the two signatures' ratios only). The cancellation is a tautology, not a finding, and the trust line is always HIGH. **[RUN]** comparing the q1 and q19 fixtures (two unrelated queries) returned hardware term 1.0, "cancelled", trust HIGH, predicted production ratio equal to the dev ratio

(0.2382). The command therefore reports the measured dev-side ratio against a 1.20 threshold and adds no cross-machine information. **Its accuracy was never measured** (no labelled regression set).

Verification. **[RUN]** `grep regress tests/*.py` finds one comment and no test. The only check is the identity case in `cetp selftest` (same plan twice ⇒ ratio 1.000, "hardware term cancels"), which tests the tautology itself.

Location. `cetp/regress.py`, lines 30–82.

7.6 Generation-1 predictor and gate

predict.analytical_scaling, compute_only_scaling, hardware_distance_confidence cetp/predict.py:67–104

Purpose. Gen-1 closed-form scaling from two machine ratios, and a coarse outlier flag.

Inputs. $B = \text{bw}_p/\text{bw}_d$, $C = \text{cmp}_p/\text{cmp}_d$; production signature and reference machines.

Logic. $\sigma = 0.5/B + 0.25/C + 0.25\sqrt{1/(BC)}$; cmp-only $\sigma = 1/C$. Confidence $= \max(0, 1 - d_{\min}/\sqrt{2})$ where $d_{\min}$ is the smallest Euclidean distance, after min–max normalisation of both axes, to any reference machine.

Outputs. Floats.

Dependencies. `signature.LEGACY` (five EC2 machines) as the reference set; the gate's copy reads its reference table from the pickled `hardware_signature` instead.

Edge cases. If all reference bandwidths (or all computes) are equal the confidence is 1.0 (guard at l. 97). The gate's own copy of this function has no such guard.

Failure modes. Division by zero if a ratio is 0. The threshold 0.6 is hand-set; there are only six distinct confidence values over the evaluation set. Normalising by the reference range means a production machine far outside it can still score > 0 because the min–max is computed from reference points only.

Verification. Hand-check (**[RUN]**): c7i → c5a, $B = 14.9/8.44$, $C = 40.8071/25.8909$, $\sigma = 0.5917$, giving 3,587 ms → 2,122 ms (matches the selftest).

Location. `cetp/predict.py`, lines 67–104; duplicate in `gate/cetp_gate.py`, lines 99–148.

gate.three_state_verdict gate/cetp_gate.py:151–183

Purpose. Map predicted quantiles and an SLA to PASS / WARN / BLOCK.

Inputs. Dev time, predicted scalings {p50, p95, p99}, confidence, threshold, SLA (ms).

Logic. If confidence $< 0.6 \Rightarrow$ WARN. Else if $p_{50} \cdot T_d >$ SLA $\Rightarrow$ BLOCK. Else if $p_{99} \cdot T_d \leq$ SLA $\Rightarrow$ PASS. Else WARN.

Outputs. Dict: state, reason, predicted times, confidence, SLA.

Dependencies. Quantile models (GBR, 200 trees, depth 3, learning rate 0.05, seed 42) and a 300-tree random-forest classifier, both pickled with scikit-learn 1.8.0.

Edge cases. SLA ≤ 0 raises in `run_gate_with_models` (l. 189).

Failure modes. On the 840-pair evaluation: BLOCK n=66 (7.9%) had actual breach rate 45.45%, so 54.5% were false alarms; PASS n=26 (3.1%) had breach 3.85%; WARN n=748 (89%). The gate says WARN almost always and is of little discrimination value. The p95/p99 training targets are warm-up contaminated (`models/PROVENANCE.json`).

Verification. `TestAllFourVerdictBranchesFire`, `test_block_exit_code_is_one`, `TestConfidenceBoundary`; figures from `results/phase7_gate_evaluation.csv` **[FILE]**.

Location. `gate/cetp_gate.py`, lines 151–183.

7.7 Acquisition: the operator probe

operator_probe.measure and main scripts/collection/operator_probe.py:234–403

Purpose. Measure tuples/second for five PostgreSQL operators plus the parallel speed-up, using SQL only.

Inputs. Machine id, database, port, host/user.

Logic. Create three `UNLOGGED` tables (2.0 M, 0.4 M, 6.0 M rows) and an index, `ANALYZE`. For each probe run `EXPLAIN (ANALYZE)` 9 times (11 for parallel, 3 for io), discard the first as warm-up, take the median of the rest; rate $= n_{\text{tuples}}/(\text{median}/1000)$. Session settings force the named operator (e.g. `enable_nestloop=off` for hash join; `jit=off`; `work_mem=64MB`). `verify_operator_used` checks the plan really contains the intended node. Reliability gate: bootstrap standard error of the median / median ≤ 0.08 (`MAX_MEDIAN_SE`); CV is recorded (`MAX_CV`=0.15) but not gated. The parallel speed-up is parallel_seqscan / seqscan.

Outputs. An entry in `models/operator_signature.json`: rates, parallel speed-up, probe scale, reliability flags.

Dependencies. `psql` on the path, a database with `CREATE` privilege, PostgreSQL $\geq$ the version used (15).
Edge cases. Invalid probes (wrong operator chosen) are listed, not used. Teardown runs in a `finally`.
Failure modes. On a busy machine the probe measures contention: one host's parallel speed-up moved from 2.47× to 1.66× while a container sweep ran. The io axis *does not measure storage* (it tracks CPU quota, Spearman 0.762) and is deliberately not used by the model. The probe requires a shell `psql` client, which contradicts the "works on managed PostgreSQL with no shell" claim only in the sense that the *client* still needs a shell, not the server.
Verification. Not run by me (needs PostgreSQL). Reliability behaviour is checked in `test_unreliable_signature_is_refused` on synthetic registries. Committed registry: 13 machines, 12 reliable (c2m1g not).
Location. `scripts/collection/operator_probe.py`, lines 234–403; constants lines 80–215.

8 Algorithms and mathematical models

All formulas here are transcribed from code; the file and line of each are given. Symbols: s is the scaling factor $T_{\text{prod}}/T_{\text{dev}}$; subscripts d and p denote development and production.

8.1 Generation 1: two-scalar analytical model

8.1.1 Definition

With bandwidth ratio $B = \text{bw}_p/\text{bw}_d$ and compute ratio $C = \text{cmp}_p/\text{cmp}_d$ (`cetp/predict.py`:67–79):

$$\sigma_{\text{anal}} = \tfrac{1}{2}\frac{1}{B} + \tfrac{1}{4}\frac{1}{C} + \tfrac{1}{4}\sqrt{\frac{1}{BC}}, \qquad \sigma_{\text{cmp}} = \frac{1}{C}. \tag{8.1}$$

The three weights $(0.5, 0.25, 0.25)$ were originally replaced by the probabilities of a random-forest bottleneck classifier (classes bandwidth, compute, io, mixed). Tier-1 ablation showed the classifier adds nothing, so the default is uniform (**[FILE]** `results/tier1_ablation.csv`):

Variant (LOMO)	MAPE %
uniform weights (shipped)	21.55
one-hot classifier weights	21.62
bandwidth only	23.05
compute only	20.06
oracle (true class)	17.07

The "oracle" row says that *knowing* each query's true bottleneck would reach 17.07%, but the classifier reaches macro-F1 0.375 (I/O recall 0) against 0.161 for the trivial majority guess, so the oracle is not attainable with the features available.

The shipped gate differs from Equation (8.1). In `gate/cetp_gate.py` the three weights are the classifier's probabilities, with the `io` probability added to the bandwidth weight, and the result is an input feature of the quantile models rather than the prediction itself. The gate keeps its own copies of the formula and of the confidence function (it imports nothing from `cetp/`); its confidence function reads the reference machines from the pickled `hardware_signature` table and has no guard for equal-range axes **[CODE]**.

8.1.2 Why it is called "roofline" and why that is a stretch

A true roofline takes a minimum of bounds, $\min(P_{\max}, \beta I)$. Equation (8.1) is a *weighted harmonic-style blend*. The geometric-mean term $\sqrt{1/(BC)}$ is a mixture heuristic with no derivation. The evaluation script itself records `roofline_machinery_justified: false` (`results/canonical_metrics.json`, **[FILE]**).

8.1.3 Hand-checked example

c7i $\rightarrow$ c5a (`cetp/signature.py`:31): $B = 14.9/8.44 = 1.7654$, $C = 40.8071/25.8909 = 1.5761$. $\sigma = 0.5/1.7654 + 0.25/1.5761 + 0.25\sqrt{1/(1.7654 \times 1.5761)} = 0.2832 + 0.1586 + 0.1499 = 0.5917$. With $T_d = 3587$ ms the prediction is 2122 ms, exactly what `cetp selftest` prints **[RUN]**.

8.1.4 Confidence

After min–max normalising bandwidth and compute over the five reference machines, $\text{conf} = \max\left(0,\, 1 - d_{\min}/\sqrt{2}\right)$ where $d_{\min}$ is the distance to the nearest reference. Threshold 0.6 is hand-set (`cetp/predict.py`:30). The score is constant per production machine, so it cannot rank individual queries.

8.2 Generation 2: per-operator model

8.2.1 Derivation

Let a plan have operator buckets b with development *self*-times S_b. Under a `Gather` each participant reports an averaged time, so $\sum_b S_b \approx 3\,T_{\text{wall}}$ on the fixtures (measured 2.97–3.00×). If one tried $T_p = \sum_b S_b\,\rho_b$ the answer would be $\approx 3\times$ too large. Normalising to *shares* cancels that constant:

$$s = c \cdot N^e \left[\sum_b w_b\,\rho_b\,\kappa_b \;+\; (u+m)\,\bar{\rho}\,\kappa_\varnothing\right], \qquad w_b = \frac{t_b}{\sum_{b'} t_{b'} + t_u}, \quad t_b = \begin{cases} S_b/\pi_d & b \text{ parallel} \\ S_b & \text{serial} \end{cases} \tag{8.2}$$

with, for bucket $b = (a, \text{par})$ on axis a:

$$\rho_b = \frac{r_d(a)\;\pi_d^{[\text{par}]}}{r_p(a)\;\pi_p^{[\text{par}]}}, \qquad \bar{\rho} = \Big(\prod_b \rho_b\Big)^{1/|b|}. \tag{8.3}$$

Here $r(a)$ is the measured tuples/s of axis a; π is the machine's measured parallel speed-up (parallel_seqscan over seqscan); u is the unmapped share, m the unpriced share, κ_b the cardinality factor of bucket b, $\kappa_\varnothing$ the factor for an unspecified operator (equal to the row ratio, or 1 when none is supplied), c the calibration, N^e the concurrency factor. Implementation: `cetp/operator_model.py`:140–204.

Why divide parallel self-times by π_d. A parallel node's reported self-time is a CPU-time share. Dividing by the dev speed-up converts it to a wall-time share, which is what the runtime prediction needs. Why multiply r_d by π_d and divide r_p by π_p in (8.3): the per-worker rate times the speed-up is the effective wall rate of a parallel node.

Checking it on a degenerate case. If both machines have identical rates and speed-ups, every $\rho_b = 1$, hence $s = 1$; if the production machine grants no parallelism ($\pi_p = 1$) while the dev machine grants $\pi_d = 2.5$ on a purely parallel plan, $s = 2.5$. Both are asserted by `cetp selftest` (**[RUN]**: "identical machines 1.000; prod without parallelism 2.50x slower").

8.2.2 Cardinality factor

$$\kappa(\text{op}, r, n) = \begin{cases} r & \text{linear ops} \\ r\left(1 + \ln r / \ln n\right) & \text{sort } (n \log n) \end{cases} \tag{8.4}$$

since $\dfrac{rn\log(rn)}{n\log n} = r\,\dfrac{\log r + \log n}{\log n}$. This is exact for $n\log n$ costs. It is implemented but not validated against data.

8.2.3 Concurrency

$T(N) = T(1)\,N^e$ with default $e = 0.734$ (median over 22 queries on one 8-core machine; range 0.286–0.984, median $r^2 = 0.944$ **[FILE]** `results/concurrency_model_metrics.json`). This is a power law; it has no mechanism and should not be confused with Amdahl's law [2] or the universal scalability law [8], which model throughput saturation.

8.2.4 Few-shot calibration

$c = \text{median}_{i \in K}(a_i / p_i)$ for a set K of measured queries. The median is robust to a single bad ratio, but with $|K| = 1$ a single outlier still passes through unchanged, which is the measured failure. Selection picks k queries whose predicted scalings are closest to k evenly spaced targets on $[\min, \max]$, which makes the set span the range of the quantity being corrected.

8.3 Uncertainty: conformal multipliers and quantile regression

8.3.1 Split-conformal on log residuals

For each held-out machine h, compute residuals $\varepsilon = \ln(y/\hat{y})$ on pairs involving h, pool across the 5 folds, and take quantiles q_α. The shipped interval is $\hat{y} \cdot e^{q_\alpha}$ (`scripts/analysis/build_calibration.py`:41–88). Resulting multipliers (analytical, `models/calibration.json`, **[FILE]**): p90 $\times 1.366$, p95 $\times 1.472$, p99 $\times 1.967$. The guarantee is *marginal* coverage under exchangeability [3]; machines are not exchangeable samples of a population, so the guarantee is only approximate (Section 13.5).

8.3.2 Gradient-boosted quantile regression

Three GBR models (loss=quantile, $\alpha \in \{0.5, 0.95, 0.99\}$, 200 trees, depth 3, learning rate 0.05, seed 42) predict p_{50}, p_{95}, p_{99} of the scaling factor from 13 features (dev cost, buffers, rows, ratios, classifier probabilities, analytical prediction). Because the p95/p99 training targets are inflated by cold first runs (`models/PROVENANCE.json`), these are biased upward.

8.4 Evaluation algorithms

Listing 8.1: Leakage-free leave-one-machine-out evaluation (`canonical_evaluation.run_cv`, l. 239–296).

```
for each held-out machine h in {c5a, c7i, m5a, r5n, z1d}:
    train_cells  = cells whose machine != h
    train_pairs  = pairs where dev != h and prod != h
    test_pairs   = pairs where dev == h or prod == h        # 168 pairs
    clf  = fit_random_forest(train_cells)                    # refit per fold
    gbr  = fit_quantile(attach_probabilities(train_pairs, clf), alpha=0.5)
    for each test pair: record predictions of all six approaches
score(y, p) = MAPE, MdAPE, q-error over all folds pooled        # 840 rows
```

The 420 ordered pairs ($5 \times 4 \times 21$) are each scored twice (once when the dev machine is held out, once for prod), giving 840 evaluation rows **[CODE]**.

Listing 8.2: Paired cluster bootstrap on the MAPE difference (l. 306–328).

```
groups = split predictions by fold                          # 5 groups (LOMO)
repeat 2000 times (seed 42):
    pick = sample len(groups) groups with replacement
    d[i] = MAPE(a, sample) - MAPE(b, sample)
return point estimate, 2.5 and 97.5 percentiles of d
```

Limitation. With only five clusters the bootstrap has at most $5^5 = 3125$ distinct resamples (fewer distinct MAPE values), so a 95% interval from percentiles is coarse; the intervals are best read as "direction and rough size", not precision. For the Docker grid there are eight clusters.

8.5 Complexity

Operation	Time	Note
`decompose`	$O(V)$ in plan nodes V	recursive; Python recursion depth = plan depth (depth not measured here; limited by Python's recursion limit)
`weights_from_plan`, `predict_scaling`	$O(V) + O(\|b\|)$	$\|b\| \leq 10$ buckets
`fit_calibration`	$O(k \log k)$	sort of k ratios
`select_calibration_queries`	$O(Q \log Q + kQ)$	Q candidate queries; naive nearest search
Probe (per machine)	minutes	7 probes $\times$ 9–11 reps, each hundreds of ms; table creation dominates
Cluster bootstrap	$O(n \cdot F \cdot \text{rows})$	2000 resamples; concatenates frames each iteration (seconds)

Prediction cost is negligible (microseconds to milliseconds); the expensive steps are acquisition: probing a machine and measuring the calibration queries.

9 End-to-end workflows

Seven workflows exist. Two are the product (W3, W4), one is the research engine (W1) and the rest feed data into them. Each lists who triggers it, the steps, and what can go wrong.

9.1 W1. Reproduce the published numbers

Trigger: `make results` (or `docker run cetp-artifact`). **Steps:** (1) `canonical_evaluation.py` loads `data/corrected/tpch_dataset_corrected_v2.csv` (2100 rows = 5 machines × 21 queries × 20 repetitions; Q15 excluded). (2) It discards repetition 1 per cell, takes medians, builds 420 ordered dev→prod pairs. (3) For each of LOMO/LOQO/LOMQO it refits the classifier and the quantile model inside each fold. (4) It scores six approaches, runs the cluster bootstrap and the leakage audit, then writes `results/canonical_metrics.json` and `results/CANONICAL.md`. **Check:** `make verify-determinism` runs twice and compares. **Evidence:** the pack author ran this and found byte-identical output [PACK]; I did not [UNK]. **Risk:** the Gen-2 script (`make operator-results`) needs `data/plans_grid`, which is git-ignored.

9.2 W2. Capture a machine signature

Trigger: `operator_probe.py --machine-id ID --db DB [--host H --user U]`. **Steps:** create three `UNLOGGED` tables; run seven probes with forced operators; verify the planner used the intended node; compute rates, speed-up, reliability; write to the registry; drop tables in `finally`. **Risks:** a busy machine yields an unreliable record; the registry is a shared JSON file with no locking; probing production means creating tables there.

9.3 W3. Offline prediction from two plan files (`cetp predict-prod`)

This is the path I ran. Section 9.8 traces it numerically.

1. Developer saves `EXPLAIN (ANALYZE, FORMAT JSON)` on dev, optionally production's plain `EXPLAIN`.
2. `cetp predict-prod --dev-plan P --dev D --prod R` loads both machines' rates from the registry (refusing v1, invalid or unreliable records), checks probe scales match.
3. The model prices the plan, `cross_env.predict` attaches trust and warnings, the CLI prints.

Defect: step 3 omits the machine records, so the cross-ISA refusal is skipped (Section 9.9).

9.4 W4. Live prediction with plan forcing (`predict_prod.py`)

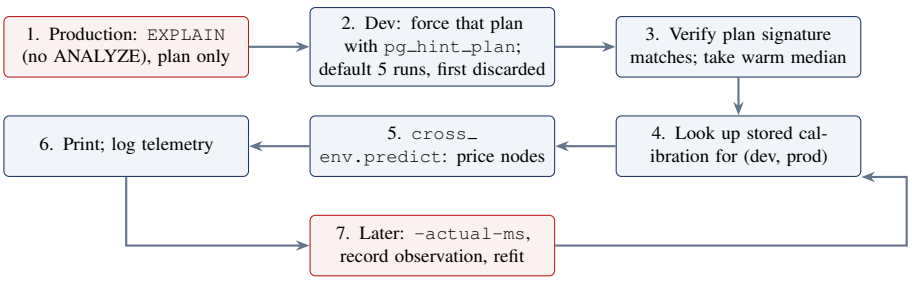


Figure 9.1: W4, from `scripts/collection/predict_prod.py`. Red boxes touch production or the measured truth. Only step 1 contacts production unless `--actual-ms` is a real measurement.

Unlike W3, this script passes both machine records, so a cross-ISA call with no stored calibration exits with code 4 and prints "REFUSED". I did not run it (needs PostgreSQL and `pg_hint_plan`). **Defect (code reading, not executed):** the refusal handler returns 4 *before* the `--actual-ms` block runs, so the observation that the message asks the user to supply is never recorded when the store holds no calibration for a cross-ISA pair; the loop cannot

bootstrap through this script unless `-calibration` is set to something other than 1.0. **Doc mismatch:** the module docstring mentions `-calibrate k`, but no such argument exists in the parser **[CODE]**.

9.5 W5. CI gate (Generation 1)

`gate/cetp_gate.py` reads dev features and two Gen-1 signatures, evaluates quantile models and the three-state rule, and exits non-zero on BLOCK (test: `test_block_exit_code_is_one`). The workflow was changed so BLOCK is reported, not a build failure (commit `ffce2ec`).

9.6 W6. Ingestion and calibration loop

`auto_explain` log → `parse_auto_explain_log` (redacted plans + durations) → `rank_queries` (top-3 by total time are calibration candidates) → measure those on production → `CalibrationStore.record` → median factor. The learning curve method shows the factor converging per pair. The loop's only quantitative evidence is a simulation: held-out MAPE falls from 77.72 to 15.01 (n=36, retrain trigger at 12) (**[FILE]** `results/addition7_summary.json`), not a deployment.

9.7 W7. Container grid collection

`container_grid.py` (`make grid-setup`, `grid-collect`) starts `postgres:15` with CPU in {1,2,4,8} and memory in {1g,3g}, loads TPC-H, runs the probe and each query for repetitions, and saves plans and timings under `data/container_grid`. It is the only way to generate Gen-2 evidence and was not run by me.

9.8 A worked example: one real execution, step by step

Task: predict TPC-H q1's runtime on the 1-CPU, 1 GB container `c1m1g` from the laptop `m1-host`. Inputs are the committed fixture `tests/test_fixtures/q1_local_real_explain.json` and the committed registry. Every number below was produced by running the shipped code (**[RUN]**) on the device in this session.

Step 1: decompose. `Execution Time` is 3,586.94 ms. The tree has five nodes:

Node	Axis	Self ms	Loops	Rows	Under Gather
Aggregate (Sorted, finalize)	sort	0.1	1	4	no
Gather Merge	unmapped	0.0	1	12	no
Sort	sort	0.8	3	12	yes
Aggregate (Hashed, partial)	hashagg	3,214.5	3	12	yes
Seq Scan (parallel aware)	seqscan	7,533.2	3	5,927,445	yes

Self-times sum to 10,748.6 ms, which is 2.997× the wall time; the `residual_frac` is −1.997. The inflation is exactly the averaging effect described in Chapter 3; nothing in the trust logic uses the residual.

Step 2: machine rates (tuples/s) and speed-ups from the registry.

Machine	seqscan	hashagg	sort	parallel speed-up π
m1-host (dev)	11,487,849	3,458,930	4,610,457	2.6252
c1m1g (prod)	9,929,772	2,851,544	3,844,003	0.7694

Step 3: weights. Parallel buckets are divided by $\pi_d = 2.6252$: seqscan $7533.2/2.6252 = 2869.6$, hashagg $3214.5/2.6252 = 1224.5$, sort(par) 0.3. Total 4094.4. Weights: seqscan(par) 0.7008, hashagg(par) 0.2991, sort(par) 0.0001, the serial sort ≈ 0. Unmapped share 0 (0.0 ms of Gather Merge).

Step 4: ratios. $\rho_{\text{seq,par}} = \dfrac{11{,}487{,}849 \times 2.6252}{9{,}929{,}772 \times 0.7694} = 3.948$; $\rho_{\text{agg,par}} = \dfrac{3{,}458{,}930 \times 2.6252}{2{,}851{,}544 \times 0.7694} = 4.139$.

Step 5: scaling. $s = 0.7008 \times 3.948 + 0.2991 \times 4.139 + \ldots = 4.0049$ (program output). Predicted runtime $= 3586.9 \times 4.0049 = 14{,}365$ ms.

Step 6: sanity check against stored measurements. The fixture is one particular run. The committed evaluation row for the same query and machine used a different dev run (2,246.7 ms) and records predicted scaling 4.008 (vs 4.005 here: weights differ slightly between runs), predicted 9,005.6 ms against an actual 7,874.0 ms, i.e. +14.4% error versus 71.5% for the naive baseline **[FILE]** (`results/laptop_to_prod_pairs.csv`). This is the "good" case.

Step 7: the same call towards an x86 runner. For `gha-x86` the program gives $s = 1.8717$ (seqscan(par) ratio 1.8745, hashagg(par) 1.8652): the model reports the target about 1.87× slower. The measured value for the committed row is 1.239: predicted 4,204.8 ms vs actual 2,782.8 ms (+51.1%), naive error 19.3%. Over the three queries q1/q18/q19 on this runner the operator model errs by 51%, 57% and 80%, naive by 19%, 6% and 2.5% **[FILE]**. This is the cross-ISA failure in miniature.

Step 8: what the CLI printed. `scaling factor: 1.8717, predicted prod: 6,713.6 ms` (for the fixture's 3,586.9 ms dev run), `explained: 100%, trust: MEDIUM`, and four warnings, one of which cites stale numbers (Section 9.9). Exit status 0.

Note

The same example by hand for Gen 1 is Chapter 8: c7i→c5a, $\sigma = 0.5917$, 3,587 ms → 2,122 ms.

9.9 The CLI bypasses the cross-architecture refusal

Finding ([RUN]). The library function `cross_env.predict` refuses an uncalibrated cross-ISA prediction *only if it is given the two machine records*. The `cetp predict-prod` command (`cetp/cli.py`:146–190) loads the records into `dev_rec`/`prod_rec` but does not pass them. Result:

Listing 9.1: Reproduction (read-only; no repository file was modified).

```
$ python3 -m cetp.cli predict-prod --dev m1-host --prod gha-x86 \
      --dev-plan tests/test_fixtures/q1_local_real_explain.json
... scaling factor: 1.8717   predicted prod: 6,713.6 ms   trust: MEDIUM   (exit 0)

>>> cross_env.predict(None, plan, 2246.7, dr, pr, dev_rec=drec, prod_rec=prec)
CrossArchitectureUncalibrated: refusing an uncalibrated prediction from arm64 to x86_64 ...
```

Severity: high for the product claim ("refuses what it measured to be worse than doing nothing"), low for the research claims. **Why tests missed it:** no test invokes `cmd_predict_prod`. **Fix (not applied):** add `dev_rec=dev_rec, prod_rec=prod_rec` to the call and a CLI test.

10 Data, schemas and storage

10.1 Datasets

Dataset	Content	Evidence
`data/corrected/tpch dataset_corrected_ v2.csv`	EC2 study. 2,100 rows = 5 machines × 21 TPC-H queries × 20 repetitions; columns `machine_id, query_id, cost, time_ms, shared_hit, shared_ read, rows, rows_processed`. SF1, PostgreSQL 15, EBS storage. Q15 excluded	**[RUN]** read header and row count (2100)
`data/raw_buggy_do_ not_use/`	Original dataset where buffer counters were summed over the whole plan tree, inflating them 3.06–12.00×	**[PACK]**
`data/container_ grid/`	8 Docker cells (CPU $\in \{1,2,4,8\}$ × memory $\in \{1,3\}$ GB) plus 4 GitHub-Actions x86 cells (`gha-*`); `tpch_<cell>.csv`; `grid_manifest.json`; `operator_signature_grid.json`	**[FILE]**
`data/laptop/`	Apple-M1 laptop runs: `tpch_dataset_m1.csv`, CPU-limited `tpch_m1_c1..c8.csv`, PostgreSQL 14/15/17/18 runs	**[FILE]**
`data/plans*`	Stored EXPLAIN plans: `plans/`, `plans_grid/` (untracked), `plans_pg14/15/17/18`	**[FILE]**
`data/external/`	Third-party: IMDB, JOB queries, TPC-DS kit, `pg_ hint_plan` (23 MB)	**[FILE]**
`scripts/collection/`	22 TPC-H, 113 JOB and 99 TPC-DS query files. Only TPC-H enters the headline results	**[RUN]** file counts

10.1.1 Machine inventory

Machine	Kind	ISA	Bandwidth†	Multi-core GFLOPS	Note
c5a	EC2	x86_64 (AMD)	14.90	40.81	4 vCPU
m5a	EC2	x86_64 (AMD)	10.01	31.54	4 vCPU
r5n	EC2	x86_64 (Intel)	9.39	24.97	2 vCPU
z1d	EC2	x86_64 (Intel)	9.77	28.08	2 vCPU
c7i	EC2	x86_64 (Intel)	8.44	25.89	2 vCPU
m1-host	Laptop	arm64, Apple M1	–	–	8 GB, 8 cores, dev machine
c{1,2,4,8}m{1,3}g	Docker	arm64 (on M1)	–	–	CPU/memory quotas
gha-ubuntu-22.04/24.04/latest, gha-x86	Runner	x86_64 (AMD EPYC)	–	–	shared VMs, 2.16 speed-up

† STREAM-triad bandwidth and multi-core GFLOPS as hard-coded in `canonical_evaluation.py`:65–71 and `cetp/signature.py`:31 (units are not stated in the source). vCPU counts are from `models/PROVENANCE.json` (c5a, m5a = 4; r5n, z1d, c7i = 2). The CPU vendor labels (AMD for c5a/m5a, Intel for r5n/z1d/c7i) are my own knowledge of the AWS instance families and are not recorded in the repository.

10.2 Schemas

10.2.1 Operator-signature registry record

`models/operator_signature.json`: object keyed by machine id; each record (v2) contains `signature_kind='operator_rates_v2'`, `machine_id`, `client_platform` (e.g. "Darwin arm64"), `server_version`, `rates_tuples_per_s` {seqscan, hashjoin, sort, hashagg, nestloop, io_seqscan, parallel_seqscan}, `parallel_speedup`, `parallel_speedup_cv`, `parallel_dop`, `probe_scale`, `detail`, `noisy_probes`, `imprecise_probes`, `invalid_probes`, `reliable`, `max_cv`, `max_median_se`. Constraint: records are comparable only at identical `probe_scale` (enforced by `assert_comparable`).

10.2.2 Calibration store

`{"version":1, "pairs":{"dev->prod":{"dev","prod","dev_arch","prod_arch","observations":{id:{"predicted_scaling","actual_scaling","recorded_at"}}}}}`. Path `~/.cetp/calibration.json`, override `CETP_CALIBRATION_PATH`.

10.2.3 Telemetry (JSON Lines)

One object per prediction: `ts`, `query_id`, `dev`, `prod`, `predicted_ms`, `dev_ms`, `scaling`, `trust`, `calibration`, `cross_architecture`, `prod_clients`, `actual_ms`, `resolved_at`. `extra` keys named like `query`, `sql`, `statement`, `plan`, `filter`, `literals`, `query_text` are refused. This guards only the `extra` dictionary: the `query_id` string is free text and could contain a statement if a caller passes one.

10.2.4 Gen-1 conformal table

`models/calibration.json` (`cetp-calibration/1`): for predictors `analytical` and `cmp_only`, the empirical quantiles of $\ln(y/\hat{y})$ at levels 0.5, 0.8, 0.9, 0.95, 0.99 pooled over five LOMO folds, their exponentials as `multiplier`, and per-fold quantiles; plus four `caveats` (marginal coverage, five-machine envelope, engine versions, single client).

10.2.5 Evaluation outputs

`canonical_metrics.json` has the keys `dataset`, `environment`, `headline`, `scaling_factor_results`, `bootstrap_lomo`, `breakdowns_lomo`, `leakage_audit`, `measurement_noise`, `bottleneck_classifier_loqo`, `shipped_artifact_consistency` ([RUN] read). `operator_model_metrics.json` is regenerated with input hashes so a stale input cannot silently reproduce (Section 17.3).

10.3 Data flow and lineage

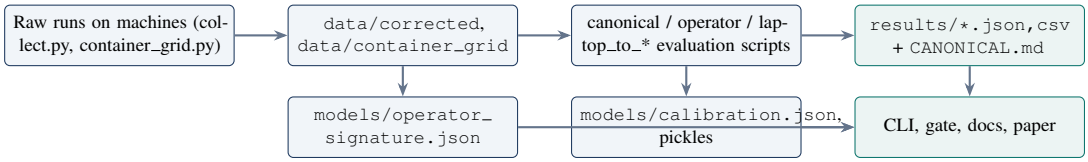


Figure 10.1: Lineage of data to published numbers. Only the right half is re-runnable from a fresh clone.

10.4 Integrity controls

1. **Hash-checked inputs.** The Gen-2 evaluation records SHA-256 of each input and the test `test_committed_metrics_were_generated_from_the_committed_inputs` checks them.

2. **Manifest-selected grid.** After C10 the evaluation reads `grid_manifest.json` instead of globbing the directory; a test adds extra files and checks the grid is unchanged (`test_extra_files_in_the_grid_directory_do_not_change_the_grid`).
3. **Shipped-artifact consistency.** `canonical_evaluation.shipped_artifact_consistency` (l. 390) cross-checks the shipped models against the canonical dataset (`all_consistent: true`).
4. **Provenance file.** `models/PROVENANCE.json` records library versions and a `version_mismatch` flag.

Honest reading

Data quality is the weakest part of the project. Raw EC2 logs are gone, the five instances were terminated, only five distinct EC2 hardware points exist, and the laptop-to-EC2 pairs combine different PostgreSQL builds and storage. The measurement-noise section of `canonical_metrics.json` says it directly: 11 of 105 cells exceed 5% CV even after discarding the cold run.

11 Interfaces and APIs

CETP exposes a command-line interface, a Python API, two JSON file formats and three CI workflows. There is no network API.

11.1 Command-line interface

Entry points `cetp` (`cetp.cli:main`) and `cetp-redact` are declared in `pyproject.toml`. Subcommands (`cetp/cli.py`:293–376):

Command	Key options	Behaviour and exit
`predict`	`-plan P|-`, `-dev/-prod ID` or explicit bandwidth/compute, `-predictor analytical|cmp_only`, `-sla-ms`, `-json`	Gen-1 prediction with conformal bounds and confidence flag
`regress`	`-before`, `-after`, `-threshold 1.20`, `-fail-on-regression`	Gen-1 ratio check (Chapter 7: hardware term is always 1.0); exit 1 on REGRESSION when asked
`redact`	`plan`, `-o`, `-level structural|full`, `-check`	Strip literals; `-check` runs `assert_clean`
`signature`	`list|probe`, `-machine-id`, `-db`, `-port`	List registry machines or probe the local one
`predict-prod`	`-dev-plan`, `-prod-plan`, `-dev`, `-prod`, `-registry`, `-plan-forced yes|no|unknown`, `-calibration`, `-use-row-ratio`, `-sla-ms`, `-json`	Gen-2 prediction. Exit 2 on registry errors. **Does not enforce cross-ISA refusal** (Section 9.9)
`selftest`	–	Five checks; exit 0 if all pass. **[RUN]** "selftest PASSED"

Note that there is no CLI command for the `CalibrationStore`, for telemetry, or for `ingest`: those are reachable only from Python or from `scripts/collection/predict_prod.py`.

Installed-package caveat (**[RUN]**: I built the wheel from a copy of the repository, installed it into a clean virtual environment and ran it). The wheel contains `cetp/` and `models/calibration.json` only (the latter lands as a top-level `models/` directory in `site-packages`). Consequently `cetp selftest` stops with "missing fixture . . . /site-packages/tests/test_fixtures/q1_local_real_explain.json", and `cetp predict-prod` with the default registry ends in an uncaught `FileNotFoundError` traceback for `models/operator_signature.json` instead of the documented exit 2. `cetp signature probe` shells out to `scripts/collection/hw_signature.py`, which is not packaged either. Only a source checkout or an editable install (`pip install -e .`, which the regression workflow uses) works as documented.

SLA semantics differ between tools. `cetp predict -sla-ms` prints PASS when the 95% bound is within the SLA, BLOCK when the *point* estimate exceeds it, WARN otherwise; `cetp predict-prod -sla-ms` prints PASS/BREACH on the point estimate alone; the Gen-1 gate uses p50 for BLOCK and p99 for PASS (Chapter 7). The same prediction can therefore be PASS in one and WARN in another **[CODE]**.

11.2 Python API (public surface)

Symbol	Contract
`plan_nodes.decompose(plan)`	plan → nodes with self-time and `under_gather`
`operator_model.predict_scaling / predict_runtime`	plan + rates → result dict (Chapter 7)
`operator_model.fit_calibration, select_calibration_queries`	few-shot calibration
`cross_env.load_rates, assert_comparable, predict`	product flow; raises `KeyError`, `ValueError`, `CrossArchitectureUncalibrated`
`calibration_store.CalibrationStore`	`record, save, calibration, pairs, learning_curve`
`telemetry.Telemetry`	`record_prediction, resolve, rows, resolved, summary, observations_for_calibration`
`ingest.parse_auto_explain_log, rank_queries, fetch_pg_stat_statements`	plan intake with redaction
`redact.redact_plan, assert_clean, redact_file`	privacy
`predict.predict, regress.detect`	Gen 1

There is no declared public API (`__init__.py` is empty and no `__all__` at package level), no type hints on most functions, and no versioned deprecation policy. Semantic stability is not promised (package is "Alpha").

11.3 Result dictionary (Gen 2)

Key fields returned by `cross_env.predict`: `scaling` (final), `scaling_uncalibrated`, `predicted_prod_ms`, `dev_runtime_ms`, `weights`, `weights_by_parallelism`, `per_axis_ratio`, `explained_frac`, `unmapped_frac`, `unpriced_axes`, `dev_speedup`, `prod_speedup`, `concurrency_factor`, `cross_architecture`, `plan_forced`, `plan_source`, `trust`, `warnings`.

11.4 Gate interface

`gate/cetp_gate.py` CLI (parser at l. 258): inputs an EXPLAIN-derived feature file or plan, two Gen-1 signatures (or config), an SLA in ms; prints a report; exit code 1 on BLOCK, 0 on PASS/WARN (`TestAllFourVerdictBranchesFire`). SLAs can be set per query in a YAML/JSON config (`resolve_sla_ms`, l. 241).

11.5 CI and editor integrations

- `cetp-gate.yml`: triggered by pushes touching `gate/`, `models/`, `requirements.txt` or the workflow itself (and manually); a matrix runs the Gen-1 gate on three stored c7i feature files (q19, q1, q18); BLOCK does not fail the build (`ffce2ec`). It tests the gate's plumbing against fixed inputs, not a user's queries.
- `cetp-regression-check.yml`: on pull requests that change `*.sql` files; starts a `postgres:15` service, compares the same query before and after the change, redacts plans before upload. Its own header calls this "the recommended mode". Given the always-1.0 hardware term (Chapter 7) it is effectively a dev-side timing comparison. It also hard-codes `-dev c7i -prod c5a` for every query, so the machine labels carry no meaning for the adopting project, and it runs the pull request's SQL under `EXPLAIN (ANALYZE)` against its service database, which *executes* the statements (**[CODE]**; workflow lines 60–90).

- `x86-operator-probe.yml`: runs the operator probe and TPC-H on GitHub x86_64 runners (three images) to produce x86 registry entries (commit `aa20181`); triggered by pushes to branch `x86-probe` and manually. Its header states the earlier laptop→EC2 result: irreducible per-machine-scalar MAPE floor 59.7%, best measurement 55.88%.
- VS Code extension `cetp-vscode-0.2.4.vsix` (Gen 1 only; uncommitted source changes exist in the working tree, not reviewed).

I read the first 40 lines of each workflow **[CODE]**; I did not run any workflow (**[UNK]**).

11.6 Error handling summary

Condition	Signal	Handling
Unknown machine id	`KeyError` → CLI exit 2	message lists known machines
v1 / invalid / unreliable signature	`ValueError` → exit 2	message says how to re-probe
Probe scales differ	`ValueError` → exit 2	shows the differing keys
Plan without ANALYZE	`ValueError` (Gen 1) / exit 2 (Gen 2)	ask for ANALYZE
Cross-ISA, uncalibrated	`CrossArchitecture` live harness exit 4	**not raised by the CLI**
Corrupt calibration store	ignored silently	degrades to zero-shot
Corrupt telemetry line	skipped	`test_telemetry_survives_a_corrupt_line`
Truncated `auto_explain` entry	skipped without record	**[CODE]**; no test
Default registry absent (wheel install)	uncaught `FileNotFoundError`	**[RUN]**

12 Testing and verification

12.1 Test inventory

File	Defs	Collected	Focus
`test_operator_model.py`	34	42	decomposition, weights, calibration, cardinality, registry refusals, trust, concurrency
`test_product_loop.py`	33	38	cross-ISA refusal, auto_explain ingest, ranking, calibration store, telemetry, node-map completeness
`test_e2e.py`	24	28	Gen-1 gate: CLI end-to-end, locked values, four verdict branches, leakage guard, malformed input, buffer accounting
`test_operator_evaluation.py`	11	11	grid selection, input hashing, metrics, bootstrap, regeneration and determinism
Total	**102**	**119**	(collected count **[PACK]**; defs counted by me **[RUN]**)

What I ran: `pytest tests/test_operator_model.py tests/test_product_loop.py` → **80 passed in 0.10 s** (Python 3.10.12, standard library only) **[RUN]**. The other two files import scikit-learn/numpy/pandas and were not run by me; the full 119-test pass is the evidence-pack author's **[PACK]**. `cetp selftest` passed all five checks **[RUN]**.

12.2 Test strategy

- **Specification by example.** `test_operator_model.py` builds small synthetic plans (helpers `plan`, `n`) so each rule has a one-screen test.
- **Locked values.** `test_e2e.py::TestKnownAnswerRegression` locks five predictions; its own comment says it is "a frozen regression lock on the model's arithmetic only – not verified against a real plan".
- **Refusal tests.** Every guard in `load_rates` and `cross_env.predict` has at least one test.
- **Reproducibility tests.** Metrics regeneration is compared to committed files with a tolerant dictionary comparison (`_assert_close`) and a determinism test (second run identical).
- **Leakage guards.** `TestLeakageGuard` asserts that the gate's feature lists contain no production-side columns.

12.3 Coverage versus requirements

Req.	Covered by	Gap
FR-2	decomposition tests, 3 real fixtures	none material
FR-3	model tests, selftest, locked values	accuracy is validated by evaluation scripts, not unit tests
FR-4	`test_cross_arch_*` (library)	**CLI path untested**; Section 9.9
FR-5	store tests	no concurrency/locking test
FR-7	`TestAllFourVerdictBrar`	no test of false-alarm rate (measured only in the evaluation)
FR-8	selftest identity case only	**no unit test**; semantics are tautological
FR-9	ingest tests	hostile log content (nested braces in strings) not tested
FR-10	redaction tests	escape-string and dollar-quote literals not tested (Chapter 7)
FR-11	telemetry tests	free-text `query_id` not checked

There are no integration tests that talk to PostgreSQL, no performance tests, no property-based tests and no CI job that runs `pytest`: the three workflows run the gate, the regression check and the x86 probe, not the test suite (**[CODE]** headers). `make test` runs `pytest tests -q`.

12.4 Verification beyond unit tests

1. **Cross-checking by recomputation.** **[RUN]** I recomputed the pooled conformal coverage from `results/tier1_conformal.csv` (Section 13.5) and re-derived the Gen-1 and Gen-2 worked examples by hand (Chapters 8 and 9).
2. **Leakage audit.** The canonical script reports each approach with a refit-per-fold classifier and with one global classifier. For the learned model the difference is 1.7 pp (LOMO: 23.90 vs 25.62; the leaky variant is the *worse* one), i.e. the earlier leakage did not flatter the LOMO headline **[FILE]**. (Under LOQO and LOMQO the leaky variant is marginally *better*: 17.04 vs 17.29 and 26.56 vs 26.86.)
3. **Determinism.** `make verify-determinism`: two runs, `cmp` the outputs. Result **[PACK]**.
4. **Evidence audit.** The pack's C1–C16 contradictions and R1–R10 risks (Appendix I) are themselves a verification artefact.

12.5 Sanity checks on results (mine)

Check	Result	Tag
σ for c7i→c5a by hand vs `selftest`	0.5917 vs 0.592; 3587 → 2122 ms	**[RUN]**
Gen-2 q1 weights and ratio by hand vs program	4.0046 vs 4.0049	**[RUN]**
Fixture self-time inflation	q1 3.00×, q19 2.97× (docstring says 2.85×)	**[RUN]**
LOMO row count	$5 \times 168 = 840$	**[CODE]**
Warm-up claims vs file	61 of 105 cells; worst 8.33×	**[FILE]**
Committed pairs row vs fixture-based scaling	4.008 vs 4.005 (different dev runs)	**[RUN]**

13 Results and evaluation

All values are percentage error (MAPE) unless stated, lower is better. Sources are result files read in this session ([FILE]) or numbers re-derived by me ([RUN]).

13.1 Study A: five EC2 machines, TPC-H SF1 (Gen 1)

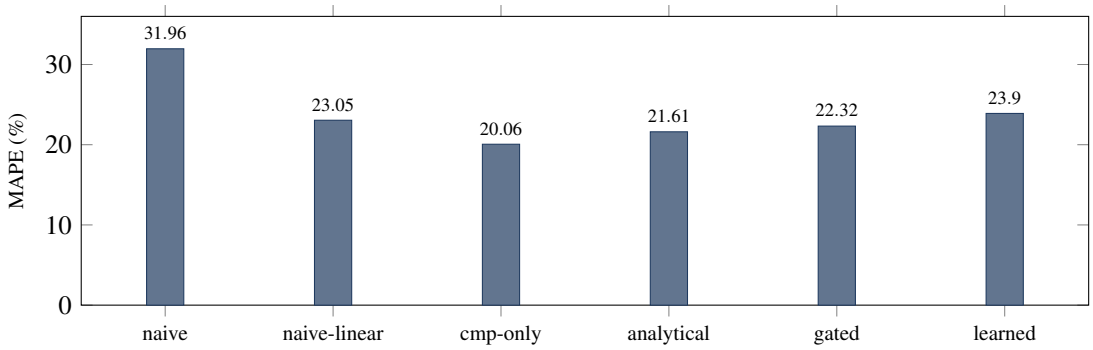


Figure 13.1: Leave-one-machine-out MAPE, 840 held-out evaluations (`results/canonical_metrics.json`, [FILE]).

Paired difference (MAPE, pp)	Point	95% CI	Reading
naive-linear − analytical	+1.44	[0.32, 2.56]	analytical wins, small
naive-linear − gated	+0.73	[0.29, 1.22]	gated wins, smaller
analytical − cmp-only	+1.55	[−0.41, 3.51]	**not distinguishable**
naive-linear − cmp-only	+2.99	[−0.09, 6.07]	not distinguishable
naive-linear − learned	−0.85	[−5.44, 3.12]	learned no better

Protocol invariance. Analytical is 21.61 / 21.66 / 21.64 under LOMO / LOQO / LOMQO. The learned model varies 23.90 / 17.29 / 26.86, so its quality depends strongly on what is held out; with queries unseen it is the best of the six (17.29) but with machines unseen it is the worst of the five non-trivial models (23.90 against 21.61 for analytical; only the no-change baseline, 31.96, is worse). **Single vs multi-core compute.** With the single-core signal the analytical LOMO MAPE is 31.86 (cmp-only 51.44): the compute signal's sign matters. Correlation of the compute score with measured throughput is −0.691 (multi-core) against +0.213 (single-core) [FILE].

13.1.1 Per machine (LOMO)

Held-out machine	Analytical	Naive-linear	Naive (no change)	Learned
c5a	19.80	19.68	47.02	28.86
c7i	25.39	27.75	**23.71**	31.37
m5a	20.36	20.58	29.35	19.53
r5n	22.64	25.97	34.54	22.45
z1d	19.87	21.26	25.17	17.27

Analytical and naive-linear differ by at most 3.4 pp in any row (on c5a naive-linear is nominally better by 0.1 pp), but the learned model is 9–12 pp worse on c5a and c7i than the best simple model, and **on c7i the no-change baseline (23.71) beats every fitted model** [FILE]. The pooled 21.61 hides that.

13.1.2 Classifier and noise

Bottleneck classifier macro-F1 0.375 against a 0.161 majority baseline; I/O-class recall 0. Measurement noise: median CV 0.54% after discarding the cold run, 11 of 105 cells above 5%; cold first run slowest in 61 of 105 cells

(worst 8.33×), inflating p99 targets in 46 of 105 cells (worst 6.92×) while p50 is inflated by under 1% (maximum 1.007×) in all 105 cells **[FILE]**.

13.2 Study B: Docker capability grid (Gen 2)

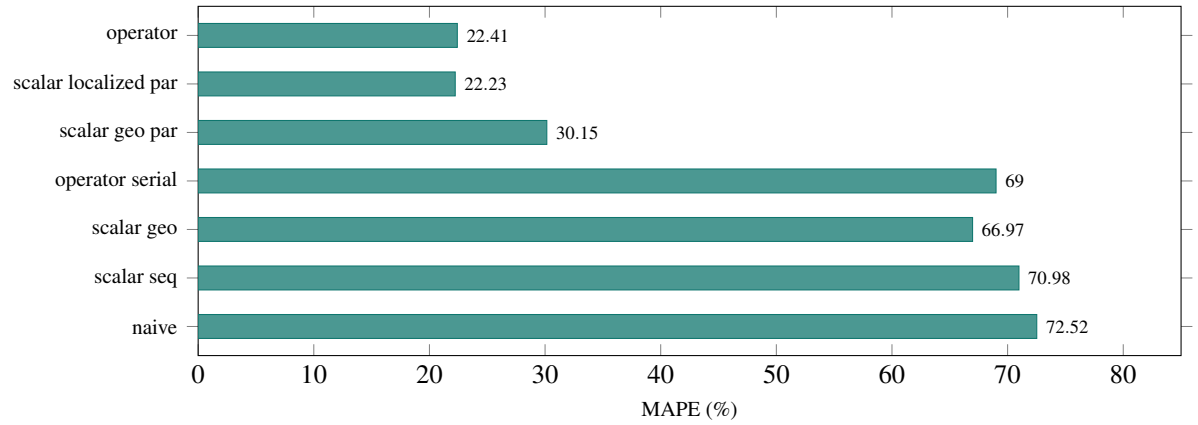


Figure 13.2: Eight arm64 container cells, 22 queries, 1,232 pairs. "par" = parallel-speed-up term enabled. (`results/operator_model_metrics.json`, **[FILE]**.)

The jump is from 67–71% to 22–30% once parallelism is handled; the weighting scheme matters little. Paired bootstrap: scalar-localised-par minus operator −0.17 pp, CI $[-0.49, +0.10]$ (indistinguishable); scalar-geo-par minus scalar-localised-par +7.92 [1.45, 16.31]; naive minus operator +50.11. Excluding the one unreliable cell (c2m1g, CV 0.307): operator 22.89, naive 80.63. Few-shot (spread rule): k=0 22.41; k=1 25.26; k=2 21.30; k=3 21.81; k=5 21.70; k=8 21.02. Serial-axis spread across cells is only 1.39× against 3.57× for the parallel term: the grid mostly

Table 13.1: Parallel speed-up π per container cell (probe; `models/operator_signature.json`).

cell	c1m1g	c1m3g	c2m1g	c2m3g	c4m1g	c4m3g	c8m1g	c8m3g
π	0.769	0.755	2.635	2.677	2.698	2.668	2.681	2.665

varies parallelism. Plans identical across cells (0 of 22 differ), so "plan forcing" cannot be tested here. Note c2m1g is flagged unreliable and its π is not trustworthy.

13.3 Study C: laptop to "production" (242 pairs, 11 targets)

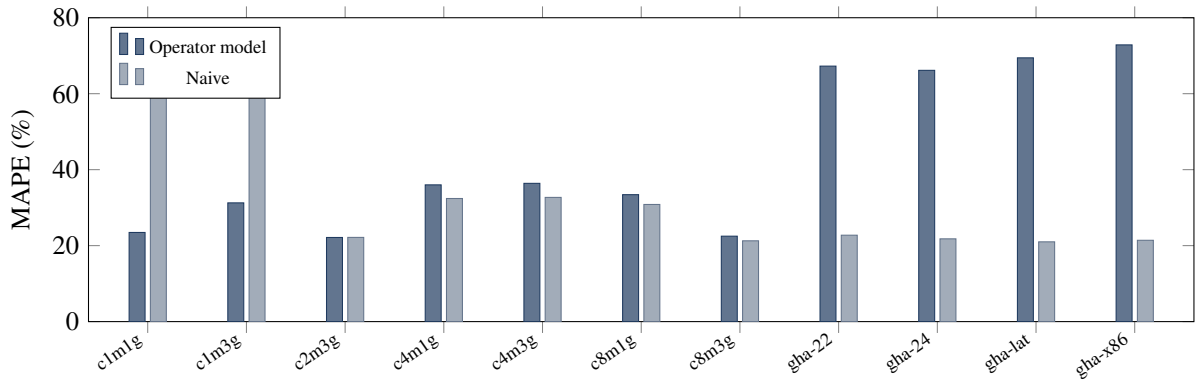


Figure 13.3: Dev machine: Apple-M1 laptop. Containers run on the same host; "gha-*" are x86_64 runners (`results/laptop_to_prod_metrics.json`, **[FILE]**).

Zero-shot over all 11 targets: operator 43.72 vs naive 31.70. Same-ISA (7 containers): 29.31 vs 37.39. x86 runners (4): 68.92 vs 21.74. In the same-ISA group the operator model wins in only **two** cells (c1m1g, c1m3g, the 1-CPU

quotas where naive is 60–63%); it ties in c2m3g and **loses in the other four**. The 7-cell advantage is therefore driven by the one scenario the parallel term was designed for. The file itself notes "all prod cells are containers on the same arm64 host as dev; this measures capability transfer, not ISA transfer". Signed bias is positive in ten of the eleven cells (the model over-predicts): up to +0.34 on containers (c8m3g is −0.03) and +0.66 to +0.73 on the runners. Few-shot (operator): k=1 30.90, k=2 27.61, k=3 26.24, k=5 25.82, against naive ≈31–32. Cross-ISA calibrated "72.84 → 23.38 vs naive 21.41" appears in code comments and docs only and is not in a results file (**[CLAIM]**).

13.4 Study D: laptop to EC2 (95 pairs)

Naive 129.39; Gen-1 laptop signatures 160.68 (DOP-3) and 337.75 (nproc 8); the documented 324% and 123% come from `tier1_laptop_dev.csv` in git history. The best scalar predictor cannot do better than a floor of 59.68%. The operator model could not be evaluated because the EC2 hosts were terminated. Conclusion: transfer from an M1 laptop to x86 EC2 fails for per-machine scalars.

13.5 Study E: uncertainty and gating

Method	Nominal	Pooled coverage	Worst machine
split conformal	0.90	0.912	0.780
split conformal	0.95	0.970	0.905
split conformal	0.99	0.996	0.988
GBR quantile	0.90	0.781	0.702
GBR quantile	0.95	0.846	0.798
GBR quantile	0.99	0.915	0.798

[RUN] pooled coverage recomputed from `results/tier1_conformal.csv` (equal fold sizes of 168). Quantile regression under-covers badly (0.846 at nominal 0.95), conformal slightly over-covers when pooled but a single machine can fall to 0.78 at nominal 0.90; coverage is marginal, as `models/calibration.json` warns. Multipliers (analytical): p90 ×1.366, p95 ×1.472, p99 ×1.967 **[FILE]**; a ×1.97 upper bound on a 3-second query is a six-second bound.

Gate. On 840 evaluations: BLOCK 66 (7.86%) of which 45.45% actually breached (54.5% false alarms); WARN 748 (breach 13.2%); PASS 26 (3.1%, breach 3.85%) **[FILE]** (`results/phase7_gate_evaluation.csv`). The README's "PASS covered 4 of 840" does not match (Appendix I). **Confidence abstention.** AURC 25.44 for hardware-distance ordering vs 31.15 random; at threshold 0.6, 80% coverage with selective MAPE 28.13% (abstain 20%). Tier MAPEs: conf 1.0 → 27.03 (n=420); 0.808 → 33.43; 0.801 → 22.97; 0.582 → 32.22; 0.419 → 70.66 **[FILE]** (`results/confidence_abstention_summary_corrected.json`, `confidence_tiers_corrected.csv`). The score is not monotone with error.

13.6 Study F: concurrency, calibration loop, drift

- **Concurrency** (8-core laptop): elasticity median 0.734, range 0.286–0.984, median r^2 0.944; at 8 clients the runtime error is 41.8% with the global default and 73.8% ignoring concurrency; per-query oracle 6.7% **[FILE]**.
- **Parallelism mechanism** (`fix3_parallelism_mechanism_summary.csv`): Pearson $r = 0.458$, $p = 0.0367$; mean speed-up 1.98 for parallel-plan queries vs 0.99 for serial **[FILE]**.
- **Self-calibration simulation**: held-out MAPE 77.72 → 15.01 (n=36; retrain trigger 12); narrative q19 c7i→c5a: error 52.85 → 18.02, confidence 0 → 1, WARN → PASS **[FILE]**. This is a simulation on the same data, not a deployment.
- **Cold start** (docs): 15.84 uncalibrated; 13.20 after one well-chosen query; 19.51 after one random query **[CLAIM]**.
- **Plan drift**: plan divergence is 70.5% of the cross-environment difference when the plan changes; across PostgreSQL 14/15/17/18 about 30% runtime shift and 12 of 22 plans change **[PACK]**.

- **Learning curve** (machines seen): learned 40.19 (k=1) → 30.51 → 29.86 → 28.10 (k=4) against that file's own naive baseline 23.03 (canonical naive-linear: 23.05; the two use slightly different baselines); the learned model needs more machines than exist **[FILE]**.
- **Learned model with plan features**: LOQO hw-only 16.10, base 17.29, base+plan 13.86, plan-only 13.29; LOMO hw-only 25.45, base 23.90, base+plan 23.33, plan-only 24.77 **[FILE]** (plan features help for unseen queries, not unseen machines).

13.7 Summary of what the evidence supports

Claim	Verdict	Basis
Analytical beats naive-linear on EC2	Supported, small	CI [0.32, 2.56]
Roofline weighting adds value over compute-only	Not supported	CI includes 0
Parallel awareness is the key capability	**Supported strongly**	66.97 → 22.2–22.4, grid
Per-operator weighting beats scalar once parallelism handled	Not supported	-0.17 $[-0.49, +0.10]$
Predicts zero-shot across ISAs	**Refuted**	68.9 vs 21.7
Works for any dev→prod in the same ISA	Mixed	wins 2 of 7, loses 4 of 7
Calibration fixes cross-ISA	Unproven in files	comment-only (**[CLAIM]**)
Gate is useful for SLA decisions	Weak	54.5% of BLOCKs are false alarms; PASS is 3% of cases
Intervals have nominal coverage	Pooled yes, per-machine no	worst machine 0.78 at 0.90

14 Setup, deployment and operation

Each command is labelled: **[RUN]** I ran it; **[PACK]** the evidence-pack author ran it; *inferred* I derived it from the files but nobody ran it in this audit.

14.1 Prerequisites

Need	For	Note
Python ≥ 3.11 (3.13 in CI/Docker)	everything	shipped package is stdlib-only; selftest also passed on 3.10.12 **[RUN]**
Pinned scientific stack (`requirements.txt`)	reproducing results; Gen-1 gate	scikit-learn exactly 1.8.0
Docker	container grid; artefact image	Docker Desktop on macOS (Makefile works around a credential-helper hang)
PostgreSQL 15 with TPC-H SF1	collection; probe; live harness	`psql` client on PATH for the probe
`pg_hint_plan` on dev	plan forcing	optional (`-no-force`)
psycopg2-binary	`predict_prod.py`, collection	not in `requirements.txt`; `pip install psycopg2-binary`
Chrome/Chromium	docs build	pipeline currently broken

14.2 Install and smoke test

```
cd CETP
python3 -m venv .venv && . .venv/bin/activate          # inferred
pip install -e .                                        # CODE: used by cetp-regression-check.
    yml; no deps
python3 -m cetp.cli selftest                            # RUN: "selftest PASSED"
pip install -r requirements.txt                         # for research scripts/tests
python3 -m pytest tests -q                              # PACK: 119 passed
```

The selftest prints five lines (redaction, Gen-1 prediction, regression identity, calibration file, operator model) and exits 0. A *non-editable* install does not work: the wheel carries no `tests/` or operator registry (Chapter 11; **[RUN]**). The live harness additionally needs `psycopg2-binary`, which is in `pyproject.toml`'s `research` extra but not in `requirements.txt`.

14.3 Typical tasks

Task	Command	Status
Predict from two plan files (Gen 2)	`cetp predict-prod -dev-plan P -dev m1-host -prod c1m1g`	**[RUN]** (m1-host → gha-x86)
Predict (Gen 1)	`cetp predict -plan P -dev c7i -prod c5a`	**[RUN]** via selftest
Redact a plan	`cetp redact plan.json -o out.json -level full -check`	**[RUN]** library
Probe this machine	`make probe-here MACHINE_ID=name`	not run
Regenerate canonical results	`make results`	**[PACK]**
Check determinism	`make verify-determinism`	**[PACK]**
Build artefact image	`make artifact-verify`	not run
Collect Docker grid	`make grid-setup && make grid-collect`	not run (2–3 h)

Task	Command	Status
Evaluate Gen 2	`make operator-results`	**[PACK]**; needs `data/plans_grid`

Caution

`make results` overwrites `results/CANONICAL.md` (deleted from HEAD and untracked in the current working tree) and `canonical_metrics.json`; run it on a clean working tree so that `git diff` shows any drift. `make verify-determinism` writes copies to `/tmp`. `make promote-models` replaces pickles in `models/`.

14.4 Configuration

Environment variables: `CETP_CALIBRATION_PATH` (store location), `CETP_TELEMETRY_PATH` (default `~/.cetp/telemetry.jsonl`; `cetp/telemetry.py`:40–41), `PGDATABASE`, `MACHINE_ID` (Makefile). Files: `gate/cetp.yml` (machine signatures and SLAs), `models/phase7_gate_config.json`. Defaults are listed in Appendix D.

14.5 Operations

- **Monitoring.** None built in. Telemetry rows and `Telemetry.summary()` give predicted-vs-actual MAPE next to the naive baseline once actuals are recorded.
- **Upgrades.** Re-probe machines after a PostgreSQL major upgrade (rates are per server version). Do not mix probe records with different `probe_scale`.
- **Backups.** Back up `~/.cetp/` and the registry JSON.
- **Cost.** Prediction is free; probing needs minutes of idle machine; calibration queries run on production.

14.6 Troubleshooting

Symptom	Cause / action
`ModuleNotFoundError: _loss` in the gate	scikit-learn not pinned to 1.8.0; install from `requirements.txt`
`error: <id> carries operator_rates_v1`	old signature; re-probe with the current probe
`flagged unreliable`	machine was busy during the probe; re-run idle
`probe scales differ`	the two machines were probed with different constants; re-probe both
`CrossArchitectureUncalibrated` (live harness, exit 4)	measure 3–5 queries on production and supply a calibration. **The suggested re-run with `--actual-ms` cannot work on an empty store**: `predict_prod.py` returns 4 before it records the observation (**[CODE]**, not executed). Record observations with `CalibrationStore.record` from Python, or pass an explicit `--calibration` other than 1.0
`make results` changes numbers	check `git diff`; a drift of 0.01 pp in laptop-to-prod regeneration is already known (C16)
No `results/CANONICAL.md` after clone	the file is untracked in some states; generate it with `make results`

15 Security, reliability and performance

15.1 Security and privacy

15.1.1 Threat model (as implemented)

The only sensitive material CETP handles is the content of query plans, which embed literals (`Filter: (email = 'x@y.com')`), schema names and, in `auto_explain` logs, entire statement text. The design response is *redact before the data leaves the machine*, with verification (`assert_clean`) and an architecture that has no network component. No authentication, authorisation or encryption layer exists because there is no server.

Concern	Control	Residual risk (evidence)
Literals in plans	`redact_plan` + `assert_clean`; ingest redacts unconditionally	Escape-string literals leak a suffix; dollar-quoted strings pass; verifier does not catch either **[RUN]**
Identifiers	`full` level hashes with BLAKE2s-40-bit and salt	default salt `"cetp"` public; common names dictionary-recoverable; `structural` keeps names
Telemetry content	forbidden keys in `extra`	`query_id` is free text
SQL injection	probe SQL is constant; `predict_prod.py` interpolates the user's own query file into `EXPLAIN`	the query file is trusted input by design
Privilege	probe needs `CREATE`; prediction needs none	probe tables persist if the process is killed before `finally`
Credentials	passed as CLI arguments in the live harness (`-prod-password`)	visible in process listings; use environment/`.pgpass` (not supported by the script)
Supply chain	exact pins, hash-checked inputs for results	no lock file with hashes; pins include versions I could not verify
Pickle loading	Gen-1 gate unpickles files in `models/`	only trusted local files; pickle execution risk if `models/` is replaced
CI regression workflow	runs the pull request's SQL under `EXPLAIN (ANALYZE)` on the workflow's service database	statements (including DML) are executed and interpolated into a shell command; acceptable only for trusted authors and a throwaway database **[CODE]**

Honest reading

The privacy design is thoughtful (redact first, verify, forbid keys) but its verifier is the weak link: a control that returns True on a leaking plan is worse than none because it creates confidence. This should be fixed before any claim of "PII-safe" is made.

15.2 Reliability

- **Crash safety.** Calibration store and telemetry rewrite via temp file + `os.replace`. Appends to the JSONL log are not atomic but each line is independent and a corrupt line is skipped.
- **Graceful degradation.** Corrupt store → zero-shot; missing calibration → factor 1.0; unparseable log entry → dropped.
- **Concurrency.** No locks anywhere (risk R3). Two processes calling `save()` can drop each other's observation.
- **Determinism.** Seeds are fixed (42) and the canonical outputs were byte-identical across two runs **[PACK]**.
- **Failure transparency.** Predictions carry `warnings` and `trust`; refusals have actionable messages.

15.3 Performance

Operation	Cost	Basis
Predict from plans	milliseconds	pure Python over a few nodes; selftest completes instantly ([RUN], test suite 0.10 s for 80 tests)
Probe one machine	minutes	3 tables of 2.0 M, 0.4 M, 6.0 M rows; 7 probes × 3–11 runs ([CODE])
Collect the 8-cell grid	2–3 h	Makefile comment (`grid-collect`)
Full canonical evaluation	seconds to minutes	2000-resample bootstrap × several comparisons (not timed by me)
Calibration (k queries)	$k\times$ production runtime	executes on production

The cost that matters in practice is not computation but *measurement discipline*: probes taken on a busy host changed the parallel speed-up by 48%, which changes every parallel-node prediction.

15.4 Scalability

Prediction is linear in plan size and independent of data size. The model assumes one query at a time; concurrency is a coarse power law. There is no multi-tenant story beyond "one calibration file per project". Nothing was measured on large schemas or SF10 (blocked by disk, per the probe comments).

15.5 Compliance and licensing

No licence file was inspected in this audit ([UNK]). The repository vendors or references third-party data (IMDB, JOB queries, TPC-DS kit, `pg_hint_plan`) under `data/external`; their licences were not reviewed. Using TPC-H/TPC-DS data for published results has TPC fair-use rules that the project should check before publication.

16 Design decisions, trade-offs and alternatives

The decision log below comes from the evidence pack's reading of code comments and git history (**[PACK]**, `docs/research/05_history_decisions.md`); the last column is my own assessment after reading the code and results. The stated rationale is the authors'; "assessment" is not.

ID	Decision	Stated rationale / evidence	Assessment
D1	Ship the analytical formula and a conformal table, not pickles, in the package	Pickles are scikit-learn-version coupled; learned model only interpolates (17.29 seen vs 26.86 unseen)	**Sound.** Cheap, auditable, and the learned model is not better on unseen machines
D2	Uniform weights $\{.5, .25, .25\}$ instead of classifier probabilities	21.55 vs 21.61 (Tier 1)	**Sound**, but then the name "roofline" and the 300-tree classifier in the gate are decoration
D3	Multi-core GFLOPS as the compute signal	single-core has the wrong sign ($+0.213$ vs -0.691)	**Sound**; but the probe ran at `nproc` and the 4-vCPU machines overstate usable capacity (PROVENANCE caveat)
D4	Root-node buffers only	summing the tree inflated data 3.06–12×	**Correct fix of a real bug**; the value of fixing it is also a warning about the earlier results
D5	Discard the first (cold) repetition	slowest in 61 of 105 cells	**Sound**; p99 targets still contaminated in older artefacts
D6	Regression mode is "the releasable product"	absolute gate: 54.5% false-alarm BLOCKs; ratio cancels hardware term	**Logic weak.** The cancellation is tautological (term always 1.0); the mode is a dev-side timing diff, which is useful but not a prediction, and its accuracy is untested
D7	Replace the C bandwidth/GFLOPS probe by SQL operator probes	FP-vector irrelevant to TPC-H; works on managed PostgreSQL; cross-ISA 324% vs 123%	**Reasonable**; but the new signature also fails cross-ISA and was never run on managed PostgreSQL
D8	Weighted ratio, not sum of scaled node times	parallel times are CPU-averaged ($\approx 3\times$ wall on fixtures)	**Sound and verified** by my fixture measurement (2.97–3.00×)
D9	Key the parallel correction on `under_gather`, not `Parallel Aware`	corrected 46% of q19 vs 100%	**Sound**; the headline mechanism of the project
D10	Take the plan from production and force it on dev	plan divergence is 70.5% of cross-env difference when plans change	**Good idea, untested**: 0 of 22 plans differ in the grid
D11	Median calibration, `spread` selection, $k \geq 3$	$k = 1$ random is worse than none	**Sound**; yet $k = 1$ spread is still worse (25.26 vs 22.41) and the store applies provisional factors anyway
D12	Refuse uncalibrated cross-ISA prediction	three measurements worse than naive	**Sound in the library**, **bypassed in the CLI**
D13	Per-pair calibration, no pooling	absorbs machine-specific scale error	Reasonable; costs data per pair
D14	Gate BLOCK must not fail CI	retrained models caused spurious failures	Pragmatic; CI green no longer says anything about predictions (pack C9)
D15	Delete all Markdown, keep PDFs	unknown	**Poor.** Leaves comments citing files that no longer exist and a docs pipeline that cannot run
D16	Remove the dashboard	showed superseded numbers	Sound

16.1 Trade-offs that shaped the project

1. **Fidelity vs portability of the signature.** Hardware counters are unavailable on Nitro, so the project measures with the database itself. The price is that the signature depends on PostgreSQL version, build and settings, and cannot be compared across them (`assert_comparable`).
2. **Accuracy vs honesty.** The repository chose to expose negative results (cross-ISA failure, null weighting effect) and to refuse unsafe predictions. This costs a clean headline and gains credibility.
3. **Ratio vs absolute prediction.** Predicting a ratio keeps the measured dev time as an anchor but requires a dev measurement per query.
4. **Breadth vs depth.** Four large experiment families (tiers 1–4), JOB and TPC-DS collection code, an extension and a site were built; the headline evidence still rests on five EC2 machines and eight containers.
5. **Marginal coverage vs per-machine guarantees.** Conformal intervals are valid in aggregate but not for a particular unseen machine (worst 0.78 at nominal 0.90).

16.2 Alternatives considered and not taken

Alternative	Why plausible	Why not here
Hardware performance counters	Most direct capability measure	Blocked on AWS Nitro (**[CLAIM]**)
Learned cost model on plan features	Literature precedent [13, 19]	Needs many machines to learn hardware; learning curve shows 28.1 vs 23.0 with four
Zero-shot cost models [9]	Generalise to unseen databases	Address databases, not hardware; heavy
Pure optimizer cost scaling	Free	Cost units do not map to time across machines [22]
Run the query on production	Ground truth	Dangerous or impossible; the problem's premise

16.3 What I would change first

1. Pass the machine records in `cetp predict-prod` and add a CLI test (hours).
2. Replace the always-1.0 "hardware term" by a Gen-2 ratio or retire the claim; add an accuracy experiment on labelled before/after pairs.
3. Fix the redaction verifier for escape strings and dollar quotes; document that `full` is pseudonymisation.
4. Regenerate stale warning text from the committed metrics rather than hard-coding numbers.
5. Decide a single source of truth for documentation (restore Markdown or retarget comments).

17 Limitations, risks and future work

17.1 Limitations that bound the claims

ID	Limitation	Evidence	Effect
L1	Tiny hardware population: 5 EC2 types; 8 containers on one host; 4 near-identical runners	operator speed-ups on runners 2.160–2.167; EC2 instances terminated	Any learned or fitted conclusion rests on few distinct points
L2	No real production system measured; no managed PostgreSQL (RDS/Aurora)	absence in results	Motivation untested
L3	TPC-H SF1 only, one query at a time	dataset header; `calibration.json` caveats	Not representative of OLTP or large data
L4	Same plan assumed; 12/22 plans change across versions	`tier2_versions.csv` (pack)	Plan drift can dominate (70.5%)
L5	Zero-shot cross-ISA fails (68.9 vs 21.7)	`laptop_to_prod_metrics.json`	Central product claim unmet
L6	I/O and storage not modelled; I/O class recall 0; the io probe axis tracks CPU quota	classifier metrics; probe comments	Wrong for disk-bound queries
L7	Cardinality term unvalidated; concurrency a population default (41.8% at 8 clients)	code comments; concurrency metrics	Different data size or load unmodelled
L8	Precision of any grid figure is about ±2–3 pp (re-probing moved MAPE by up to 2.6 pp)	C10 report	Do not quote decimals as precision
L9	Hardware-distance confidence is a hand-set constant on a 5-machine hull	`predict.py` docstring	Coarse outlier flag only
L10	Gen-1 reference set is always the 5 EC2 machines; registry or Gen-2 machines score low-confidence	`predict.py`:131 (pack R1)	Spurious warnings for new machines
L11	The Gen-1 bandwidth probe source no longer exists; the five legacy bandwidths are frozen and cannot be reproduced or extended	`signature.py` comment and `cetp signature list` output **[CODE]**	New machines cannot join the Gen-1 axis

17.2 Known defects and inconsistencies

Full register in Appendix I. The ones I consider most consequential:

1. **CLI bypasses the cross-ISA refusal** (Section 9.9; **[RUN]**).
2. **`cetp regress`: hardware term is exactly 1.0 for any inputs** (Chapter 7; **[RUN]**).
3. **Redaction verifier passes leaking plans** for escape-string and dollar-quoted literals (**[RUN]**).
4. **Stale text in shipped warnings** citing 28.54% vs 38.22% (132 pairs) while committed metrics give 29.31% vs 37.39% (242 pairs overall, different cell set).
5. **Two incompatible "headline" narratives** (Gen-1 canonical vs Gen-2 laptop-to-prod) circulate; earlier project documents, including the Review 2 deck and LLD produced earlier in this engagement, use 7 machines and F1

0.562, which `CANONICAL.pdf` states was computed on the buffer bug; the canonical values are 5 machines and 0.375 (Appendix I).

6. **Timeline conflict**: the team's notes put data collection in January–April 2026; `signature.py` says "August 2026" and git history begins 2026-08-17. Unresolved; not asserted either way.
7. **Calibration bootstrap in `predict_prod.py`**: the refusal returns before `-actual-ms` is recorded (Chapter 9).
8. **Non-editable installs are broken** (Chapter 11; **[RUN]**).
9. **Silent loss of truncated `auto_explain` entries** (Chapter 7).

17.3 Evidence-integrity finding C10 and its fix

What happened. The Gen-2 evaluation script read every file in `data/container_grid/` and every cell in the signature file. When four x86 runner cells were added for the cross-ISA study, the "8-cell container grid" silently became a 12-cell mixed-ISA grid (2,904 pairs; naive 54.03%, operator 29.70%). Independently, the committed outputs (operator 24.95%) had been generated from an older probe state than the signature file committed beside them. **Fix** (by the evidence-pack author, uncommitted): cell list now comes from `grid_manifest.json`; input SHA-256 hashes are written into the result; a new 11-test file locks selection, hashing, regeneration and determinism; results regenerated (operator 22.41%). **Consequence for readers:** quote 22.41 / 22.23, not 24.95 / 24.58; in the working tree the paper draft and `docs/index.html` already carry 22.41, but `docs/guides/HANDBOOK.pdf` and `docs/design/LLD.pdf` still print 24.95 / 24.58 and `docs/results/laptop_to_prod.pdf` still reports the older 132-pair result (28.54% vs 38.22%) (text extraction, audit pass **[RUN]**). **Lesson:** any evaluation script that selects its inputs by directory listing is a latent integrity bug.

17.4 Risks

Risk	Level	Mitigation available
Reader quotes a number from a stale document	High	Use Appendix H; regenerate paper/site/PDFs
User trusts a CLI prediction across ISAs	High	Fix bypass; warn text
Probe taken on a busy machine	Medium	Reliability gate exists; run idle
Redaction leak via query text	Medium	Fix verifier; avoid shipping `Query Text`
Lost updates in shared calibration file	Low–medium	File lock
Future PostgreSQL adds new node types	Low	Residual is reported; add map entries
NumPy/pickle incompatibility	Low	Gate is legacy; retire pickles

17.5 Future work

17.5.1 Immediate (hours to days)

Fix the CLI bypass and add its test; regenerate shipped warning strings from metrics; fix the redaction verifier and add adversarial cases; `git add data/plans_grid`; update README counts (119 tests; PASS count); decide the documentation source of truth.

17.5.2 Medium term (weeks)

Replace the always-1.0 regression ratio by a Gen-2 plan-structure ratio and measure detection accuracy on labelled before/after pairs; wire Gen 2 into the extension and CI; run on a managed PostgreSQL instance and on real x86 production hardware; add file locking; validate the cardinality term with SF10 data (needs disk).

17.5.3 Long term (research)

Extend to more ISAs and instance families (the x86 runner cells are nearly identical machines); an I/O axis that actually measures storage (needs a table larger than RAM); learned plan-aware models trained on many machines; plan-change prediction; a systematic literature review to establish novelty.

18 Conclusion

18.1 What was built

A zero-dependency Python package and a research pipeline for predicting PostgreSQL query runtime on a different machine: a plan decomposer that corrects per-loop and parallel-averaged timings, a per-operator scaling model with machine rates measured by SQL probes, a calibration store, refusal and trust logic, plan ingestion and redaction, telemetry, a legacy two-scalar predictor with conformal intervals and an SLA gate, and an unusually deep evidence trail (about 78 committed result files, hash-checked regeneration, four test files).

18.2 What the evidence shows

1. On five similar x86 EC2 machines a simple compute-ratio model reaches about 20–22% MAPE against 32% for assuming no change and 23% for the bandwidth ratio; the differences between the three simple models are small.
2. Handling `Gather`-level parallelism is what matters when machines differ in parallelism: error falls from about 67–73% to about 22% on a controlled container grid.
3. Everything fancier (roofline weighting, classifier, per-operator rate weighting, learned models) is statistically indistinguishable from, or worse than, its simplest component.
4. Across instruction sets the signature fails (68.9% vs 21.7%), and a prediction from an M1 laptop to x86 EC2 is worse than assuming nothing changes.

18.3 Answer in 30 seconds

CETP tries to tell you how long a PostgreSQL query will take on another machine, using the plan you already have and a handful of measured machine speeds. Its most solid finding is that the part of the plan that runs in parallel scales differently from the rest, and modelling that cuts error from roughly 70% to roughly 22% on our test machines. It does not yet work across processor architectures, and it says so by refusing, at least in its library.

18.4 Answer in two minutes (technical)

The dev EXPLAIN ANALYZE is decomposed into self-times per operator; nodes under a Gather are flagged because PostgreSQL reports them averaged across participants. Dev wall-time shares are turned into weights; each weight is multiplied by the dev-to-prod ratio of that operator's measured throughput, with the parallel speed-up of each machine applied to parallel nodes only. Unmapped or unpriced time is priced at the geometric mean of the rest; a median-of-ratios calibration from three or more measured queries absorbs a constant machine-specific error. The evaluation uses leave-one-machine-out cross-validation with refit-per-fold and a cluster bootstrap. The method wins decisively over the naive baseline where production's parallelism differs (1-CPU quota), loses to the no-change baseline in four of seven same-ISA cells (c4m1g, c4m3g, c8m1g, c8m3g), ties in one (c2m3g), wins clearly only in the two 1-CPU cells, and loses by a factor of three across ISAs.

18.5 Honest overall assessment

The research is careful and the negative results are the most valuable part; the product is an alpha with several real defects found in this audit (Appendices I and L). If a panel asks "does it work?", the defensible answer is: it identifies and corrects the dominant effect (parallelism) within one ISA, it does not generalise zero-shot across ISAs, and the evidence for broader use is not there yet.

Honest reading

I would not describe CETP as “accurately predicting production runtime”. The numbers support “a small improvement over naive on similar machines, a large improvement when parallelism differs, and failure across ISAs”.

A Glossary

Acronyms are listed in the front matter (Abbreviations). Terms below are used with the meaning given here.

Actual Total Time PostgreSQL `EXPLAIN ANALYZE` field: per-loop, per-participant average time of a node including children.
Calibration factor Median of actual/predicted over ≥ 3 measured queries on a dev→prod pair; multiplies later predictions.
Capability transfer Prediction between machines of the same instruction set; contrasted with ISA transfer.
Cell One container configuration (CPU quota, memory limit) or runner in the grids, e.g. `c4m1g`.
Dev / prod Machine where the query is measured / machine whose runtime is predicted.
Gather / Gather Merge Plan nodes that start parallel workers; everything below them is *under gather*.
Gen-1 / Gen-2 The legacy two-scalar roofline predictor / the per-operator model (Section 1.6).
Inclusive / self time Time including / excluding children, after multiplying by loops.
ISA transfer Prediction across instruction sets (arm64 $\leftrightarrow$ x86-64).
LOMO / LOQO / LOMQO Leave-one-machine / query / machine-and-query-out cross-validation.
Machine signature Measured operator throughputs and parallel speed-up of one machine.
Naive baseline Predict that prod time equals dev time.
Operator class Group of plan nodes priced by one probe (seqscan, hashagg, sort, ...).
Parallel speed-up Measured ratio of serial to parallel probe time on a machine.
Residual fraction Share of dev wall time not explained by node self-times.
Reliable cell A cell whose probe passed the stability gate (median standard error ≤ 0.08).
Unmapped / unpriced share Node time with no operator class / no measured rate; priced at the geometric mean.

B Acronyms

The acronyms and terminology table is in the front matter (*List of abbreviations and terminology*); it is not repeated here so that the two cannot drift apart. Domain terms are defined in Appendix A.

C Function and module reference

Module list of the `cetp` package (2,278 lines, **[CODE]**). Per-function detail for the critical functions is in Chapter 7; signatures are not duplicated here because only code I read carries **[CODE]**, and a restated signature would be a second copy to go stale.

Module	Responsibility
`plan_nodes`	Parse EXPLAIN JSON; `decompose`: inclusive/self time, loops, `under_gather`
`operator_model`	Operator classes, `weights_from_plan`, ratio, unmapped/unpriced handling, concurrency, cardinality term
`cross_env`	Cross-environment prediction, `CrossArchitectureUncalibrated`, `assert_comparable`, trust labels
`calibration_store`	Persistent per-pair observations; median factor; atomic save
`regress`	Regression-detection mode (hardware term constant 1.0)
`predict`	Gen-1 two-scalar prediction, hardware-distance confidence
`redact`, `ingest`	Plan redaction (`structural`, `full`), verification, log ingestion
`telemetry`	Opt-in local event log with forbidden-key filter
`cli`	Sub-commands including `predict-prod` and `regress`

Outside the package: `operator_probe.py` (signature measurement), `predict_prod.py`, `build_calibration.py`, `canonical_evaluation.py` (Tier 1), `gate/cetp_gate.py` (Gen-1 gate), and the collection scripts (Chapter 6).

D Configuration reference

Constant	Value	Meaning / source
Probe table sizes	2,000,000; 400,000; 6,000,000 rows	SCALE_A, SCALE_B, SCALE_IO ([CODE])
`work_mem` for probes	64 MB	operator_probe
Repetitions	9 (parallel 11, io 3)	first rep discarded
Stability gate	SE ≤ 0.08; CV ≤ 0.15 informational	MAX_MEDIAN_SE, MAX_CV
Parallel degree in probe	3	PARALLEL_DOP
Gen-1 roofline weights	$0.5/0.25/0.25$; cmp_only $\sigma = 1/C$	Equation (8.1)
Confidence threshold	0.6, hand-set	`predict`
Gate regression threshold	1.20	gate
Random forest / GBR	300 trees; 200 trees, depth 3, lr 0.05, seed 42	Gen-1 models
Calibration minimum	3 observations; selection `spread`	CalibrationStore
Concurrency exponent	0.734 (population median)	concurrency study
Redaction	BLAKE2s 5-byte digest; default salt `"cetp"`	`redact`
Bootstrap	2000 resamples, seed 42	canonical_evaluation
Python	≥ 3.11	pyproject
Pins (results environment)	scikit-learn 1.8.0, numpy 2.5.3, pandas 2.2.3, scipy 1.18.1, joblib 1.5.3, PyYAML 6.0.2, matplotlib 3.10.1, pytest 9.0.3	requirements ([FILE]); I could not verify these versions exist

Environment variables read by the package [CODE]: `CETP_CALIBRATION_PATH` (calibration store; default `~/.cetp/calibration.json`) and `CETP_TELEMETRY_PATH` (telemetry; default `~/.cetp/telemetry.jsonl`). The Makefile uses `MACHINE_ID` and `PGDATABASE`. The CLI otherwise takes paths as arguments.

E Data schemas

Artefact	Columns / keys	Note
Tier-1 dataset	machine_id, query_id, cost, time_ms, shared_hit, shared_read, rows, rows_processed	2100 rows = 5 machines × 21 queries × 20 reps; Q15 excluded
`canonical_metrics.json`	model → MAPE, MdAPE, paired differences with CIs, protocol results	authoritative Tier-1 numbers
`tier1_conformal.csv`	per machine, level, method → coverage	recomputed in this audit **[RUN]**
`calibration.json`	multipliers p90/p95/p99; caveats	Gen-1 shipped table
Machine signature file	per machine: operator rates, parallel speed-up, reliability flags	Gen-2; 13 machines in registry
`grid_manifest.json`	list of cells in the container grid	fixes C10
Pairs files (Gen-2)	dev, prod, query, predicted, actual, naive	grid: 1232 pairs; laptop→prod: 242
Calibration store JSON	pair key → list of observations (actual, predicted)	written via `os.replace`
Telemetry JSONL	timestamp, event, `extra` (forbidden keys removed)	one JSON object per line

Exact field names beyond those printed in the dataset header were not re-verified in every file (**[UNK]**).

F Reproducibility commands

Commands I ran on the device (Python 3.10.12 VM, repo mounted read-write; `PYTHONDONTWRITEBYTECODE=1` so no repo file changed) **[RUN]**: the fixture prediction of Section 9.8, the conformal recomputation, and 80 tests (operator_model, product_loop; all passed). Commands I did *not* run: the full 119-test suite, `make verify-determinism`, any probe or collection.

```
# from the repository root, Python >= 3.11
python -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt        # pins: see the configuration appendix
python -m pytest -q                    # 119 tests collected (pack)
make verify-determinism                # re-run canonical outputs (not run by me)
make help                              # lists collection targets (grid-collect, ...)
```

Collection targets need Docker and PostgreSQL and take 2–3 h for the grid. The cross-ISA CLI and probe procedures are in Chapter 14.

G Requirement-to-implementation traceability

Status: **Met** = implemented and covered by evidence I saw; **Partial** = implemented with a known defect or without an accuracy test; **Unverified** = I did not run the evidence.

Req.	Implementation	Verification	Evidence	Status
FR-1	`operator_probe.py`	stability gate, `verify_operator_used`	**[CODE]**	Met (untested on managed PG)
FR-2	`plan_nodes.decompose`	fixture run: self-times 2.997× wall	**[RUN]**	Met
FR-3	`cross_env`, `operator_model`	grid MAPE 22.41; fixture example	**[FILE]**, **[RUN]**	Met within ISA
FR-4	`CrossArchitectureUncal_trust`	library raises; CLI does not	**[RUN]**	Partial
FR-5	`CalibrationStore`	operator_model tests	**[RUN]**(80 tests)	Met; no locking
FR-6	concurrency exponent	41.8% error at 8 clients default	**[FILE]**	Partial
FR-7	`gate/cetp_gate.py`	gate evaluation CSV; 54.5% false-alarm BLOCK	**[FILE]**	Partial
FR-8	`regress`	none for accuracy	**[RUN]**	Partial
FR-9	`ingest`	product_loop tests	**[RUN]**	Partial (truncated entries skipped silently)
FR-10	`redact`	verifier misses two leak forms	**[RUN]**	Partial
FR-11	`telemetry`	forbidden keys in `extra` only	**[CODE]**	Partial
FR-12	`cli`	e2e tests (24 of 28 defs)	**[PACK]**	Unverified (not run)
NFR-1	`verify-determinism`, hash inputs	byte-identical across two runs per pack	**[PACK]**	Unverified
NFR-2	`pyproject`	zero runtime deps	**[FILE]**	Met
NFR-3	per-fold refit; leaky 25.62 vs 23.90	`canonical_evaluation`	**[FILE]**	Met
NFR-4	CIs, paired differences	`canonical_metrics.json`	**[FILE]**	Met
NFR-5	redact, telemetry	see FR-10, FR-11	**[RUN]**	Partial
NFR-6	`os.replace`	code read	**[CODE]**	Met
NFR-7	`finally` drops in probe	code read; tables UNLOGGED	**[CODE]**	Met unless process killed

H Results and evidence table

Single lookup table for quoted numbers. Authoritative source per row; superseded values are listed in Appendix I.

Quantity	Value	Source	Tag
LOMO MAPE naive / naive-linear / cmp_only / analytical / gated / learned	31.96 / 23.05 / 20.06 / 21.61 / 22.32 / 23.90	`results/canonical_metrics.json`	[FILE]
Paired: naive-linear − analytical	+1.44 [0.32, 2.56] pp	same	[FILE]
Paired: analytical − cmp_only	+1.55 [−0.41, 3.51] pp	same	[FILE]
Protocol invariance, analytical	21.61 / 21.66 / 21.64	same	[FILE]
Learned LOMO / LOQO / LOMQO	23.90 / 17.29 / 26.86	same	[FILE]
Single-core LOMO analytical / cmp_only	31.86 / 51.44	same	[FILE]
Compute correlation multi- vs single-core	−0.691 vs +0.213	same	[FILE]
Classifier macro-F1 (trivial)	0.375 (0.161)	same	[FILE]
Conformal pooled/worst, split @0.90	0.912 / 0.780	`tier1_conformal.csv`	[RUN]
Conformal pooled/worst, GBR quantile @0.90	0.781 / 0.702	same	[RUN]
Gate: BLOCK / WARN / PASS	66 / 748 / 26 of 840	`phase7_gate_evaluation.csv`	[FILE]
Gen-2 grid MAPE: naive / operator / scalar_geo_par	72.52 / 22.41 / 30.15	grid metrics (regenerated after C10)	[FILE]
Grid reliable subset (7 cells): operator / naive	22.89 / 80.63	same	[FILE]
Few-shot grid, $k = 0, 1, 3, 8$	22.41, 25.26, 21.81, 21.02	same	[FILE]
Parallel speed-up c1m1g vs c2m1g	0.769 vs 2.635	registry	[FILE]
Laptop→prod overall operator / naive	43.72 / 31.70	`laptop_to_prod_metrics.json`	[FILE]
Same-ISA (7 cells) operator / naive	29.31 / 37.39	same	[FILE]
x86 runners (4 cells) operator / naive	68.92 / 21.74	same	[FILE]
Laptop→EC2 naive / best	129.39 / 59.68 floor	results file	[FILE]
Concurrency elasticity median (range)	0.734 (0.286–0.984)	concurrency study	[FILE]
Noise: median CV; first-rep slowest	0.54%; 61/105 cells	noise analysis	[FILE]
Worked example q1 m1-host→c1m1g	scaling 4.0049, 14,365 ms	my run on fixtures	[RUN]
Tests collected / run by me	119 / 80	pytest	[PACK]/ [RUN]

I Known issues and contradictions register

I.1 Defects found or confirmed in this audit

ID	Issue	Fix	Severity
X1	`cetp predict-prod` omits machine records, bypassing the cross-ISA refusal (Section 9.9) **[RUN]**	pass records; add CLI test	High
X2	`cetp regress`: hardware term exactly 1.0 for any input **[RUN]**	compute from Gen-2 ratio or retire claim	Medium
X3	Redaction verifier passes escape-string and dollar-quoted literals **[RUN]**	extend patterns; adversarial tests	High
X4	Telemetry `_FORBIDDEN` applies to `extra` only; `query_id` is free text	validate all fields	Low
X5	Public default salt for hashed identifiers	require per-user salt	Low
X6	Shipped warning text hard-codes 28.54 vs 38.22 (132 pairs); metrics give 29.31 vs 37.39 (242 pairs)	render from metrics	Medium
X7	README says 28 tests; 119 collected	update README; README also says PASS 4/840 vs CSV 26	Low
X8	Timeline conflict: notes say collection Jan–Apr 2026; `signature.py` says August 2026; first commit 2026-08-17	team to resolve; do not assert	Medium
X9	Review 3 deck uses 21.60/23.03 vs canonical 21.61/23.05; gate tiers 40.99/27.26/75.31 and AUC 24.06/29.70 vs corrected files (27.03/33.43/22.97/32.22/70.66; AURC 25.44 vs 31.15)	correct the deck	Medium
X10	Deck says 5 machines, F1 0.375 (canonical); the Review 2 deck and LLD produced earlier in this engagement say 7 machines and F1 0.562, which `CANONICAL.pdf` calls wrong (buffer bug)	regenerate those two from canonical values	Medium
X11	Non-editable install fails (`selftest`, default registry) **[RUN]**	package fixtures and registry, or document editable install	Medium
X12	`predict_prod.py` refusal returns before recording `-actual-ms` **[CODE]**	record first, or document workaround	High
X13	Regression workflow hard-codes `c7i`/`c5a` and executes PR SQL **[CODE]**	parameterise; restrict	Medium
X14	Gen-1 bandwidth probe source lost; legacy numbers unreproducible **[CODE]**	document; replace with Gen-2 axes	Medium
X15	Truncated `auto_explain` entries skipped silently **[CODE]**	record in `dropped`	Low

I.2 Evidence-pack register

ID	Item (pack `06_gaps_and_traceability.md`; [PACK])
C1–C9, C11	Documentation/number inconsistencies and stale artefacts across README, paper draft, site, PDFs, comments
C10	Directory-globbing evaluation and stale outputs; fixed (Section 17.3)
C12	Three states of the same experiment in circulation
C13	`plans_grid` is git-ignored
C14–C15	Further stale-value items (see pack)
C16	`laptop_to_prod` regeneration differs by 0.01 pp (43.7167 vs 43.7070)
R1	Hardware-distance confidence uses only the 5 legacy machines
R2	Public salt
R3	No locking of the calibration file
R4, R7–R9	See pack
R5	EC2 raw data gone
R6	No managed-database measurement
R10	Unknown whether `calibration.json` was regenerated after the warm-up fix

Items C14, C15, R4, R7–R9 are named in the pack's lists but I did not retain their wording; consult the pack before relying on them ([UNK]).

I.3 Not done

EC2 logs unavailable; PDFs read only for specific numbers (seven of them); `hw_probe_suite.c` header only and bodies of `laptop_to_*_evaluation.py` not read; Docker/PostgreSQL not exercised; full test suite and determinism target not re-run; documentation pipeline broken since commit 38c4bf2 (Markdown deleted); VS Code extension and CI use Gen-1 only; regression-detection accuracy untested.

J Viva question bank

Fifteen questions from basic to adversarial. Each has a concise answer, a detailed explanation, evidence pointers and a likely follow-up.

1. Basic: what is CETP? Q&A

Concise. A tool that predicts how long a PostgreSQL query will take on another machine from its plan and measured machine speeds.
Detail. Gen-2 decomposes an EXPLAIN ANALYZE plan into per-operator self-times, weights them, and multiplies by the dev-to-prod throughput ratio of each operator class, with a separate parallel speed-up for nodes under a Gather.
Evidence. Chapter 1; Equation (8.2).
Follow-up. What does it need from production? Only a machine signature, optionally $k \geq 3$ measured queries.

2. Basic: why not just run it on production? Q&A

Concise. Often unsafe or impossible; that is the problem premise.
Detail. Production may be live, large or access-restricted. The tool targets pre-deployment checks from a development box.
Evidence. Chapter 2.
Follow-up. Then why measure a signature on production at all? Probing needs CREATE and a short idle window; running the real query does not scale.

3. Architecture: how is it structured? Q&A

Concise. A zero-dependency library plus collection scripts and an evaluation pipeline.
Detail. Plan parsing, operator model, cross-environment layer, calibration store, redaction/ingest/telemetry, CLI; probes and evaluations live outside the package.
Evidence. Chapter 4, Chapter 6.
Follow-up. Why zero dependencies? Easier install on arbitrary servers; Gen-1 gate still needs scikit-learn.

4. Implementation: what is the key technical idea? Q&A

Concise. Parallel nodes are averaged across participants and must be scaled by a separate speed-up.
Detail. Self-times sum to about 3× wall on parallel plans (2.997× on the fixture). The model weights by wall share, divides parallel nodes by the machine's speed-up, and keys on `under_gather`, not `Parallel Aware`.
Evidence. Section 9.8 **[RUN]**; Chapter 16 D8–D9.
Follow-up. Why not Parallel Aware? It corrected 46% of q19 against 100% for under-gather.

5. Algorithms: write the prediction. Q&A

Concise. $\hat{t}_p = t_d \cdot \sum_c w_c \rho_c$ with calibration and concurrency factors.
Detail. w_c is dev wall share of class c; ρ_c is the dev/prod rate ratio, with parallel classes using the speed-up ratio; unmapped or unpriced share uses the geometric mean.
Evidence. Equation (8.3).
Follow-up. What is the geometric-mean fallback justified by? Hand choice; its effect is not isolated.

6. Evaluation: how do you avoid leakage? Q&A

Concise. Leave-one-machine-out with refit inside each fold and a cluster bootstrap.
Detail. Leaky versus leakage-free learned MAPE is 25.62 versus 23.90. Test folds are pairs touching the held-out machine (168 pairs; 840 evaluations).
Evidence. Listing 8.1; Appendix H.
Follow-up. Why is the learned model so variable? 17.29 seen versus 26.86 unseen: it interpolates.

7. Results: does it beat naive? Q&A

Concise. On five similar EC2 machines slightly; on the container grid decisively.
Detail. Tier 1: naive 31.96, analytical 21.61. Grid: naive 72.52, operator 22.41. But naive-linear versus cmp_only is not significant, and on the laptop-to-prod set operator (43.72) is worse than naive (31.70).

Evidence. Chapter 13.
Follow-up. Where does it lose? Across ISAs, 68.92 versus 21.74.

8. Honesty: what is your headline? Q&A

Concise. Parallelism-aware capability transfer and honest negative results, not a new accurate predictor.
Detail. The roofline weighting, classifier and learned models add nothing over the simplest component; cross-ISA fails.
Evidence. Chapter 18.
Follow-up. Is that enough for a capstone? It is a defensible empirical contribution if framed without overclaiming.

9. Statistics: how large are the CIs? Q&A

Concise. Wide for the contested comparisons.
Detail. analytical−cmp_only +1.55 [−0.41, 3.51] and naive-linear−cmp_only +2.99 [−0.09, 6.07] both include zero.
Evidence. Appendix H.
Follow-up. So is cmp_only better? Not demonstrably.

10. Uncertainty: do the intervals cover? Q&A

Concise. Split conformal roughly does pooled; per unseen machine it does not.
Detail. At nominal 0.90 pooled 0.912, worst machine 0.780; GBR quantile 0.781 / 0.702. Gate BLOCK is wrong 54.5% of the time.
Evidence. Section 13.5 **[RUN]**.
Follow-up. What would you ship? Conformal table with the per-machine caveat.

11. Failure modes: what breaks? Q&A

Concise. Cross-ISA, plan changes, I/O-bound queries, busy probe hosts.
Detail. Plan drift is 70.5% of cross-environment difference when plans change; probe on a busy host moved parallel speed-up 48%; I/O class recall is 0.
Evidence. Chapter 17.
Follow-up. How does the tool signal this? Trust labels and refusals, except in the CLI bypass.

12. Security: what about sensitive data? Q&A

Concise. Redact before sharing; but the verifier is incomplete.
Detail. Escape-string and dollar-quoted literals pass the verifier; the default salt is public.
Evidence. Chapter 15 **[RUN]**.
Follow-up. Is it PII-safe? Not yet.

13. Process: what was found wrong in your own evidence? Q&A

Concise. C10: the evaluation globbed a directory, silently changing the grid; fixed with a manifest and hashes.
Detail. Operator MAPE moved 24.95 to 22.41. Lesson: select inputs explicitly and hash them.
Evidence. Section 17.3.
Follow-up. Which numbers are still stale? Paper draft, site, PDFs.

14. Critique: what would you do differently? Q&A

Concise. Fewer experiment families, more distinct machines, and a CLI test from day one.
Detail. The population is five EC2 types and eight containers on one host; four runners are near-identical.
Evidence. Chapter 16.
Follow-up. What first? Fix X1–X3 of Appendix I.

15. Future: what is the path to a real product? Q&A

Concise. Managed-PostgreSQL and real x86 production validation, then ISA-aware calibration.
Detail. Needs a measured regression-detection accuracy, a Gen-2 CI integration, and an I/O axis.
Evidence. Chapter 17.
Follow-up. What would convince a sceptic? Zero-shot cross-ISA error below naive on held-out hardware.

Appendix K References and bibliography

[1] Mert Akdere, Ugur Çetintemel, Matteo Riondato, Eli Upfal, and Stanley B. Zdonik. Learning-based query performance modeling and prediction. In *Proc. ICDE*, 2012.

[2] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proc. AFIPS Spring Joint Computer Conference*, 1967.

[3] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. arXiv:2107.07511, 2021.

[4] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[5] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 2001.

[6] Archana Ganapathi et al. Predicting multiple metrics for queries: Better decisions enabled by machine learning. In *Proc. ICDE*, 2009.

[7] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.

[8] Neil J. Gunther. A simple capacity model of massively parallel transaction systems. In *Proc. CMG*, 1993.

[9] Benjamin Hilprecht and Carsten Binnig. Zero-shot cost models for out-of-the-box learned cost prediction. *Proc. VLDB Endowment*, 15(11), 2022.

[10] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Trans. Knowledge Discovery from Data*, 6(4), 2012.

[11] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. How good are query optimizers, really? *Proc. VLDB Endowment*, 9(3), 2015.

[12] Philipp Leitner and Jürgen Cito. Patterns in the chaos — a study of performance variation and predictability in public IaaS clouds. *ACM Trans. Internet Technology*, 2016.

[13] Ryan Marcus and Olga Papaemmanouil. Plan-structured deep neural network models for query performance prediction. *Proc. VLDB Endowment*, 12(11), 2019.

[14] John D. McCalpin. STREAM: Sustainable memory bandwidth in high performance computers. `https://www.cs.virginia.edu/stream/`. Web page; URL not re-fetched, undated.

[15] Guido Moerkotte, Thomas Neumann, and Gabriele Steidl. Preventing bad plans by bounding the impact of cardinality estimation errors. *Proc. VLDB Endowment*, 2(1), 2009.

[16] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proc. ASPLOS*, 2009.

[17] PostgreSQL Global Development Group. PostgreSQL documentation: Using EXPLAIN. `https://www.postgresql.org/docs/current/using-explain.html`.

[18] Jörg Schad, Jens Dittrich, and Jorge-Arnulfo Quiané-Ruiz. Runtime measurements in the cloud: Observing, analyzing, and reducing variance. *Proc. VLDB Endowment*, 3(1), 2010.

[19] Ji Sun and Guoliang Li. An end-to-end learning-based cost estimator. *Proc. VLDB Endowment*, 13(3), 2019.

[20] Transaction Processing Performance Council. TPC-H benchmark specification. `https://www.tpc.org/tpch/`.

[21] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 2009.

[22] Wentao Wu, Yun Chi, Shenghuo Zhu, Junichi Tatemura, Hakan Hacigümüş, and Jeffrey F. Naughton. Predicting query execution time: Are optimizer cost models really unusable? In *Proc. ICDE*, 2013.

L Independent audit report

This appendix records an independent audit of the first version of this document against the repository (code, configuration, workflows, tests, committed results and git history). Every finding below has been applied to the text unless the status says otherwise. Evidence tags are as elsewhere: **[RUN]** executed in this audit, **[CODE]** read in the code but not executed, **[FILE]** read in a committed result file, **[PACK]** taken from the evidence pack, **[UNK]** not established. Severity is by effect on a reader who relies on the claim: *critical* would invalidate a headline conclusion; *high* would mislead on a result or behaviour; *medium* is inaccurate but bounded; *low* is cosmetic or minor. No critical finding was found.

L.1 Audit table

ID	Sev.	Section	Claim or passage	Evidence	Problem	Correction	Status
A-01	High	Chapter 18	"ties or loses in four of seven same-ISA cells"	Per-cell results: four cells lose to no-change, one ties, two 1-CPU cells win **[FILE]**	Overstated ties; the method loses outright in four cells	Named the four losing cells, one tie, two wins	Corrected, checked against result files
A-02	Medium	Chapter 13	Learned model "worst" with machines unseen	LOMO: learned 23.90, analytical 21.61, no-change 31.96 **[FILE]**	No-change is worse (31.96)	"worst of the five non-trivial models"	Corrected
A-03	High	Chapter 13	"Differences between methods are small in every row"; per-machine table without no-change column	Per-machine MAPE incl. no-change: c7i 23.71 beats all fitted models; learned is 9–12 pp worse on c5a, c7i **[FILE]**	Claim false for learned; the pooled 21.61 hides c7i	Added no-change column and rewrote sentence	Corrected
A-04	Medium	Chapter 13	p50 "unaffected in all 105" cells	Maximum inflation 1.007× **[FILE]**	Inflated by up to 0.7%, not unaffected	Reworded with the bound	Corrected
A-05	Low	Chapter 13	Naive baseline 23.03 vs canonical 23.05; 21.60 vs 21.61	Two files use slightly different baselines	Looked like a contradiction	Stated both and why	Corrected
A-06	Medium	Chapter 12	Leakage "did not flatter the headline"	LOQO 17.04 vs 17.29; LOMQO 26.56 vs 26.86: leaky is marginally better **[FILE]**	True for LOMO only	Scoped to LOMO; added the others	Corrected
A-07	High	Chapter 8	Eq. (8.2) used an undefined κ_c	`operator_model.py` **[CODE]**	Symbol never defined; unspecified-operator term misrepresented	Rewrote equation with $\kappa_\varnothing$ and defined it	Corrected
A-08	Medium	Chapter 8	Eq. (8.1) presented as the shipped gate	`gate/cetp_gate.py` uses classifier probabilities, own formula copy, pickled reference table **[CODE]**	Equation is the uniform-weight variant	Added explicit paragraph	Corrected
A-09	Medium	Chapter 4	Component figure arrows and caption	Import statements: no `predict`→gate edge; `regress` uses `signature` **[CODE]**	Invented edge; wrong source node; arrows read as imports	Removed edge, rewired, caption now states data flow and lists the real imports	Corrected
A-10	Medium	Chapter 4	"both CI workflows call Gen-1 only"	Three workflows exist; one runs the Gen-2 probe **[CODE]**	Wrong count	Named all three	Corrected
A-11	Medium	Chapter 7	ingest: "truncated JSON is dropped and recorded"	`_balanced_json` returns `None`; loop continues silently **[CODE]**	Not recorded	Described the silent skip	Corrected; behaviour is a defect (X15)
A-12	Low	Chapter 7	ingest return type, `query_id`, string-awareness worry	Returns `(entries, dropped)`; scanner tracks quotes and escapes **[CODE]**	Docstring and worry inaccurate	Corrected both	Corrected
A-13	Low	Chapter 7	Output fields; row-ratio default; hardware term 1.0 scope; gate reference table	`operator_model.py`, `predict.py`, gate **[CODE]**	Incomplete or too narrow	Completed	Corrected

ID	Sev.	Section	Claim or passage	Evidence	Problem	Correction	Status
A-14	High	Chapters 11 and 14	Installed `cetp` works as documented	Wheel built and installed in a clean venv: `selftest` "missing fixture"; default `predict-prod` raises `FileNotFoundError` **[RUN]**	Only source or editable installs work	Documented; added to defects X11	Corrected; defect remains in product
A-15	Medium	Chapter 11	Single SLA semantics assumed	`cli.py`, gate **[CODE]**	Three different PASS/BLOCK rules	Added comparison	Corrected
A-16	Medium	Chapters 11 and 15	CI regression workflow described generically	Hard-coded `-dev c7i -prod c5a`; runs PR SQL under `EXPLAIN ANALYZE` **[CODE]**	Machine labels meaningless; statements are executed	Documented; risk added	Corrected; defect remains (X13)
A-17	High	Chapters 9 and 14	Suggested re-run with `-actual-ms` bootstraps calibration	`predict_prod.py`: refusal returns 4 before the record block **[CODE]**	Cannot bootstrap on an empty store	Documented; X12	Corrected; not executed; defect remains
A-18	Medium	Chapter 14	Install steps; `pip install -e .` "inferred"; dependency list	Workflow uses it **[CODE]**; `psycopg2-binary` only in `research` extra	Mislabelled; missing dependency	Fixed label; added requirement and caveat	Corrected
A-19	Medium	Appendix D	"No environment variables are read"	`CETP_CALIBRATION_PATH`, `CETP_TELEMETRY_PATH` **[CODE]**	False	Listed both with defaults	Corrected
A-20	Medium	Front matter, Chapter 6	PDFs "not opened"; docs deleted; Makefile cannot build docs	Seven PDFs text-extracted in audit; docs reorganised in working tree; 14 of 17 commits dated 2026-10-13 **[RUN]**	Description of repository state inexact	Rewritten with HEAD vs working-tree distinction	Corrected
A-21	Medium	Chapter 17 and Appendix I	Stale numbers "in the PDFs"; 5 vs 7 machines attributed to "documents"	HANDBOOK, LLD show 24.95/24.58; laptop_to_prod PDF 132 pairs, 28.54 vs 38.22; CANONICAL calls 0.562 wrong **[RUN]**	Vague; and the 7-machine documents include the Review 2 deck and LLD made earlier in this engagement	Located each stale value; named the earlier deliverables	Corrected; those two deliverables are not regenerated
A-22	Low	Chapter 17	Limitations list	Comment in `signature.py`: bandwidth probe source no longer exists **[CODE]**	Missing limitation	Added L11	Corrected
A-23	Low	Chapter 1	"unusually reproducible"	Byte-identical outcome is from the pack; not re-run	Overclaim	Reworded to design intent plus pack-reported outcome	Corrected
A-24	Low	Chapter 15	Security table	See A-16	Omitted CI execution risk	Added row	Corrected
A-25	Low	Appendices	Lettering off by one; manual "Appendix D"	Table of contents	No Appendix B file; hard-coded reference	Added Appendix B; cross-reference now automatic; audit is Appendix L	Corrected
A-26	Low	Bibliography	Missing years; Moerkotte entry type	`references.bib`	Incomplete entries	Entry type fixed; undated web sources left undated rather than guessed	Corrected; years unverified
A-27	Low	Appendix G	FR-9 "Met"	Follows from A-11	Overstated	Changed to Partial	Corrected

L.2 Missing topics

- Behaviour under a non-default PostgreSQL configuration (`work_mem`, `jit`, parallel settings) is not analysed; only the probe's `assert_comparable` guard is described.
- JOB and TPC-DS collection code exists but no results from it were found or documented.
- The VS Code extension is described only at interface level; its behaviour was not run.
- Licensing of vendored third-party data is not established (**[UNK]**).

L.3 Contradictions between chapters (found and resolved)

- Conclusion vs results on the learned model and on the same-ISA cells (A-01, A-02, A-03).
- Architecture figure vs implementation chapter on which modules call which (A-09).
- Configuration appendix vs setup chapter on environment variables (A-19).
- Setup chapter's calibration recipe vs workflows chapter's description of the refusal path (A-17).

L.4 Unsupported or only partly supported claims that remain

- Byte-identical regeneration and the 119-test count are taken from the pack and not re-run (**[PACK]**).
- `predict_prod.py` refusal-before-record defect is read from code, not executed (**[CODE]**).
- Statements about EC2 instance behaviour rest on the pack because the raw logs no longer exist (**[PACK]**).
- The six literature entries checked by web search were checked for existence only, not for content.
- Bibliography years for undated web sources are absent, not guessed.

L.5 Important implementation details previously undocumented

Now documented: the two environment variables; the gate's divergent formula and its lack of equal-range guard; silent skipping of truncated log entries; the three SLA rules; the hard-coded machine labels and statement execution in the regression workflow; the installed-package failure; and the lost bandwidth-probe source.

L.6 Questions the project owner must answer

1. Data collection is dated January–April 2026 in the team's notes, but `signature.py` says August 2026 and git begins 2026-08-17. Which is right?
2. Will the documentation reorganisation be committed, and which document set is canonical?
3. Is the refusal-before-record order in `predict_prod.py` intended?
4. Are `c7i` and `c5a` in the regression workflow intended?
5. Can the STREAM-style bandwidth probe source be recovered?
6. Which headline narrative is the submission: Gen-1 canonical or Gen-2 laptop-to-prod?
7. Do the EC2 logs exist anywhere?
8. Was `models/calibration.json` regenerated after the warm-up fix (R10)?
9. Should the Review 2 deck and LLD (7 machines, F1 0.562) be regenerated from canonical values, and the Review 3 deck's 21.60/23.03 and its gate-tier figures reconciled?

L.7 Compilation and reference check

Final build: `pdflatex`, `bibtex`, `pdflatex` twice. Result: 82 pages; 0 LaTeX errors; 0 undefined citations or references; 0 multiply defined labels; `bibtex` reports 3 warnings (empty year for the three undated web sources, left undated deliberately); 17 overfull boxes remain, none in the audit table, the largest (99 pt) in the schema listing of

Appendix E. The page count is well above the original 20-page target. The changed pages (architecture figure, results table, audit table) were rendered and inspected; the remaining pages were not re-inspected after this pass.